Task : Compare a Variable with a Numeric Value

Outcome

The script checks whether the variable count is equal to 100.

Expected output:

count is 100

Objective

To understand how to use variables and if-else conditional statements in Bash.

Prerequisites

- Access to an Amazon Linux EC2 instance.
- Basic knowledge of Linux commands and Bash scripting.
- Access to the working directory.

Step-by-Step Implementation

Step 1: Create the script

vim variable.bash

Step 2: Add the following script

```
#!/bin/bash


count=100


if [ "$count" -eq 100 ]
then
   echo "count is 100"
else
   echo "count is not 100"
fi
```

Step 3: Save the file and provide execute permissions

chmod 755 variable.bash

Step 4: Execute the script

./variable.bash

Validation Steps

Verify that the output is:

count is 100

The initial attempt displayed the error [: missing]. After correcting the conditional syntax, the script executed successfully.

Evidence:

```
[root@ip-172-31-31-187 bash]# cd assignment-02
[root@ip-172-31-31-187 assignment-02]# vim variable
[root@ip-172-31-31-187 assignment-02]# mv variable variable.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 variable.bash
[root@ip-172-31-31-187 assignment-02]# ./variable.bash
./variable.bash: line 4: [: missing `]'
count is not 100
[root@ip-172-31-31-187 assignment-02]# vim variable
[root@ip-172-31-31-187 assignment-02]# vim variable.bash
[root@ip-172-31-31-187 assignment-02]# ./variable.bash
count is 100
[root@ip-172-31-31-187 assignment-02]# cat variable.bash
#!/bin/bash

count=100
if [ $count -eq 100 ]
then
   echo count is 100
else
   echo count is not 100
fi
```

Explanation

- #!/bin/bash specifies the Bash interpreter.
- count=100 assigns the value 100 to the variable.
- if checks the condition.
- -eq compares two numeric values.
- then executes the commands when the condition is true.
- else executes when the condition is false.
- fi ends the conditional statement.
- echo displays the message.

Important: Spaces are required inside the test brackets.

Real-Time Use Case

Conditional statements are useful for validating configuration values, checking deployment conditions, and automating administrative tasks.

Common Errors and Troubleshooting

Error	Solution
[: missing]	Add spaces before the closing bracket.
Unexpected output	Verify the variable value and numeric comparison operator.

Permission denied Run chmod 755 variable.bash.

Task 2: Check Whether a File Exists

Outcome

The script checks whether /home/iafzal/error.txt exists.

Expected output depends on the file:

File exists

or

File does not exist

Objective

To understand file-existence testing using conditional statements.

Prerequisites

- Access to a Linux terminal.
- A Bash interpreter.
- Knowledge of Linux file paths.

Step-by-Step Implementation

Step 1: Create the script

vim error.bash

Step 2: Add the script

```
#!/bin/bash


if [ -e /home/iafzal/error.txt ]
then
   echo "File exists"
else
   echo "File does not exist"
fi
```

Step 3: Provide execute permissions

chmod 755 error.bash

Step 4: Execute the script

./error.bash

Validation Steps

The screenshot shows the following output:

File does not exist

This indicates that the specified path was not found at the time of execution.

You can verify the path with:

ls -l /home/iafzal/error.txt

Evidence:

```
File does not exist
[root@ip-172-31-31-187 assignment-02]# vim variable02.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 variable.bash
[root@ip-172-31-31-187 assignment-02]# ./variable.bash
count is not 100
[root@ip-172-31-31-187 assignment-02]# chmod 755 variable02.bash
[root@ip-172-31-31-187 assignment-02]# ./variable02.bash
Today is not Monday
[root@ip-172-31-31-187 assignment-02]# date
Fri Oct 23 13:08:36 UTC 2026
[root@ip-172-31-31-187 assignment-02]# cat variable02.bash
#!/bin/bash

a=`date | awk '{print $1}'`

if [ "$a" == Mon ]

        then
        echo Today is $a
        else
        echo Today is not Monday
fi
```

Explanation

- -e checks whether a filesystem path exists.
- if evaluates the condition.
- echo prints the appropriate message.
- else handles the case where the path does not exist.

To check specifically for a regular file, use -f instead of -e.

Real-Time Use Case

A deployment script can check whether a required configuration file exists before starting an application.

Common Errors and Troubleshooting

Error	Solution
File does not exist	Verify the file path using ls -l.
Incorrect path	Update the script with the correct absolute path.
Need to check a regular file	Replace -e with -f.

Task 3: Check Whether Today Is Monday

Outcome

The script checks the current weekday and displays an appropriate message.

The screenshot shows:

Today is not Monday

Objective

To understand command substitution, extracting date information, and using conditional statements.

Prerequisites

- Linux terminal access.
- Bash interpreter.
- System date and time configured correctly.

Step-by-Step Implementation

Step 1: Create the script

vim variable02.bash

Step 2: Add the script

```
#!/bin/bash


a=$(date +%a)


if [ "$a" = "Mon" ]
then
   echo "Today is $a"
else
   echo "Today is not Monday"
fi
```

Step 3: Provide execute permissions

chmod 755 variable02.bash

Step 4: Execute the script

./variable02.bash

Step 5: Verify the system date

date

Validation Steps

The screenshot shows the server date as Friday, October 23, 2026, and the script prints:

Today is not Monday

```
File does not exist
[root@ip-172-31-31-187 assignment-02]# vim variable02.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 variable.bash
[root@ip-172-31-31-187 assignment-02]# ./variable.bash
count is not 100
[root@ip-172-31-31-187 assignment-02]# chmod 755 variable02.bash
[root@ip-172-31-31-187 assignment-02]# ./variable02.bash
Today is not Monday
[root@ip-172-31-31-187 assignment-02]# date
Fri Oct 23 13:08:36 UTC 2026
[root@ip-172-31-31-187 assignment-02]# cat variable02.bash
#!/bin/bash

a=`date | awk '{print $1}'`

if [ "$a" == Mon ]

        then
        echo Today is $a
        else
        echo Today is not Monday
fi
```

Explanation

- date +%a returns the abbreviated weekday, such as Mon, Tue, or Fri.
- $(...) captures the output of a command and stores it in a variable.
- a stores the weekday.
- = compares two strings.
- The script prints a different message depending on the weekday.

The original script used date | awk '{print $1}'. Using date +%a is simpler and avoids depending on the default output format of date.

Real-Time Use Case

This type of script can be used to run maintenance activities only on specific days of the week.

Common Errors and Troubleshooting

Error	Solution
Incorrect weekday output	Verify the server date using date.
Conditional syntax error	Ensure spaces exist around [and].
Variable contains unexpected text	Use date +%a to extract the weekday directly.

Task 4: Read User Input and Respond

Outcome

The script asks for the user's name and whether they like working in IT.

Example output:

What is your name?

naveen

Hello naveen sir

Do you like working in IT? [y/n]

y

You are cool

Objective

To understand how to accept user input and use that input in a conditional statement.

Prerequisites

- Access to an interactive Linux terminal.
- Bash interpreter.

Step-by-Step Implementation

Step 1: Create the script

vim input.bash

Step 2: Add the script

```
#!/bin/bash

read -r -p "What is your name? " name

echo
echo "Hello $name sir"

read -r -p "Do you like working in IT? [y/n] " answer

if [[ "$answer" =~ ^[Yy]$ ]]
then
```

```
    echo "You are cool"
else
    echo "Thanks for your answer"
fi
```

Step 3: Save and execute the script

```
bash input.bash
```

Alternatively, make the script executable:

```
chmod 755 input.bash
./input.bash
```

Validation Steps

Enter a name such as naveen and answer y when prompted. Confirm that the script displays the greeting and You are cool.

```
what is your name?
naveen
hello naveen sir
do you like working in IT? [y/n]
y
you are cool
[root@ip-172-31-31-187 assignment-02]#
```

Explanation

- read accepts input from the user.
- -p displays a prompt.
- name stores the user's name.
- answer stores the yes/no response.
- [[...]] evaluates the condition.
- =~ checks the response against a pattern.
- echo displays the result.

The pattern ^[Yy]$ accepts either uppercase Y or lowercase y.

Real-Time Use Case

Interactive scripts can collect deployment parameters or ask an administrator for confirmation before performing an operation.

Common Errors and Troubleshooting

Error	Solution
Empty name	Enter a name when prompted.
Unexpected response	Check the input and the conditional expression.
Script does not prompt	Run it from an interactive terminal.

Task 5: Create and delete Multiple Files Using a for Loop

Outcome

The script creates five files named 1, 2, 3, 4, and 5.

Expected output when confirmation messages are included:

File 1 created

File 2 created

File 3 created

File 4 created

File 5 created

Objective

To understand the for loop and create multiple files automatically.

Prerequisites

- Bash interpreter.
- Write permission in the working directory.

Step-by-Step Implementation

Step 1: Create the script

vim for-loop.bash

Step 2: Add the script

```
#!/bin/bash


for i in {1..5}
do
  touch "$i"
  echo "File $i created"
done
```

Step 3: Provide execute permissions

chmod 755 for-loop.bash

Step 4: Execute the script

./for-loop.bash

Step 5: Verify the created files

ls -l 1 2 3 4 5

Delete the file

Step 1: Create the script

vim for-loopdlt.bash

Step 2: Add the script

```
#!/bin/bash
for i in {1..5}
do
  rm $i
done
```

Step 3: Provide execute permissions

chmod 755 for-loopdlt.bash

Step 4: Execute the script

./for-loopdlt.bash

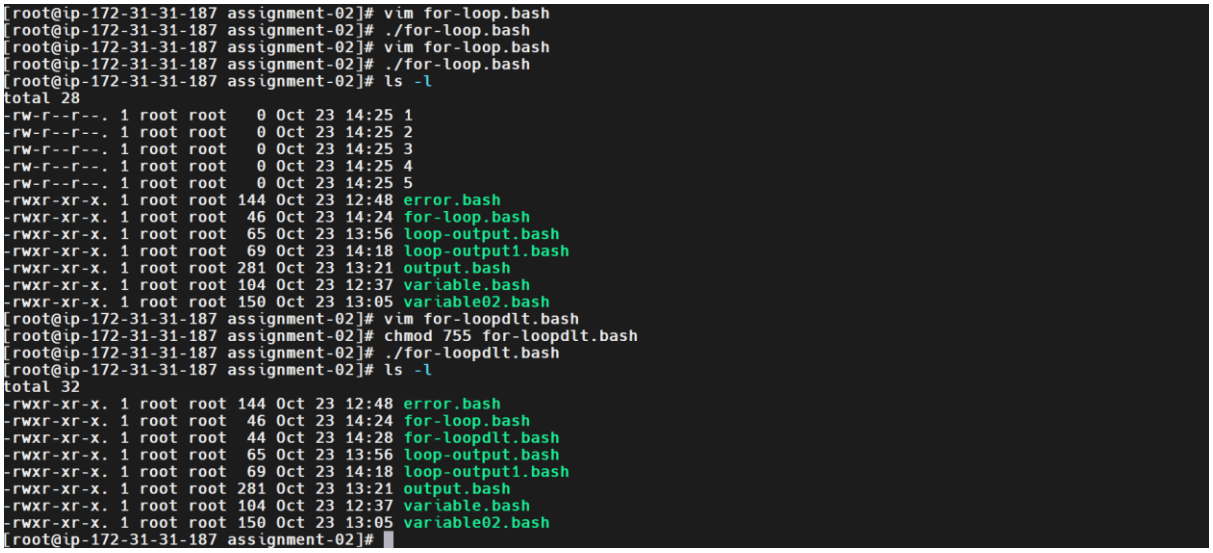
Step 5: Verify the deleted files

ls -l

Validation Steps

The screenshot confirms that files 1 through 5 were created and deleted in the working directory.

The original script did not display any output because touch is silent by default. Adding echo displays a confirmation message for each file.



Explanation

- for starts the loop.
- i is the loop variable.
- {1..5} generates the numbers 1 to 5.
- do starts the loop body.
- touch "$i" creates an empty file if it does not exist.
- echo displays a confirmation message.
- done ends the loop.

Note: If a file already exists, touch updates its timestamp. It does not delete or replace the file's contents.

Real-Time Use Case

Loops are useful for creating multiple files, directories, or test resources during automation.

Common Errors and Troubleshooting

Error	Solution
No output appears	touch is silent; add echo or verify using ls -l.
Permission denied	Check the current directory and write permissions.
Files are created in the wrong location	Run pwd to verify the working directory.

Task 6: Print a Message Repeatedly Using a for Loop

Outcome

The script prints a welcome message five times.

Expected output:

welcome 1 times

welcome 2 times

welcome 3 times

welcome 4 times

welcome 5 times

Objective

To understand how a for loop executes a command repeatedly.

Prerequisites

- Linux terminal access.
- Bash interpreter.

Step-by-Step Implementation

Step 1: Create the script

vim loop-output.bash

Step 2: Add the script

```
#!/bin/bash


for i in {1..5}
do
   echo "welcome $i times"
done
```

Step 3: Provide execute permissions

chmod 755 loop-output.bash

Step 4: Execute the script

./loop-output.bash

```
[root@ip-172-31-31-187 assignment-02]# vim loop-output.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 loop-output.bash
[root@ip-172-31-31-187 assignment-02]# ./loop-output.bash
welcome 1 times
welcome 2 times
welcome 3 times
welcome 4 times
welcome 5 times
[root@ip-172-31-31-187 assignment-02]# vim loop-output1.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 loop-output1.bash
[root@ip-172-31-31-187 assignment-02]# ./loop-output1.bash
./loop-output1.bash: line 4: syntax error near unexpected token `echo'
./loop-output1.bash: line 4: `echo "see imran $i"'
[root@ip-172-31-31-187 assignment-02]# vim loop-output1.bash
[root@ip-172-31-31-187 assignment-02]# ./loop-output1.bash
see imran eat
see imran run
see imran play
see imran jump
[root@ip-172-31-31-187 assignment-02]#
```

Validation Steps

Confirm that five lines appear, numbered from 1 to 5. The screenshot shows the expected output.

Explanation

The loop iterates through numbers 1 to 5. During each iteration, the value of i changes and is displayed in the message.

Real-Time Use Case

Repeated loops can be used in testing, batch operations, and executing the same command against several resources.

Common Errors and Troubleshooting

Error	Solution
Syntax error near echo	Ensure do appears before the loop commands.
Loop does not finish correctly	Verify that done is present at the end.
Output appears only once	Check the loop range {1..5}.

Task 7: Loop Through a List of Words

Outcome

The script prints a message for each word in the list.

Expected output:

see imran eat

see imran run

see imran play

see imran jump

Objective

To understand how a for loop processes a list of values.

Prerequisites

- Bash interpreter.
- Linux terminal access.

Step-by-Step Implementation

Step 1: Create the script

vim loop-output1.bash

Step 2: Add the script

#!/bin/bash

for i in eat run play jump

do

  echo "see imran $i"

done

Step 3: Provide execute permissions

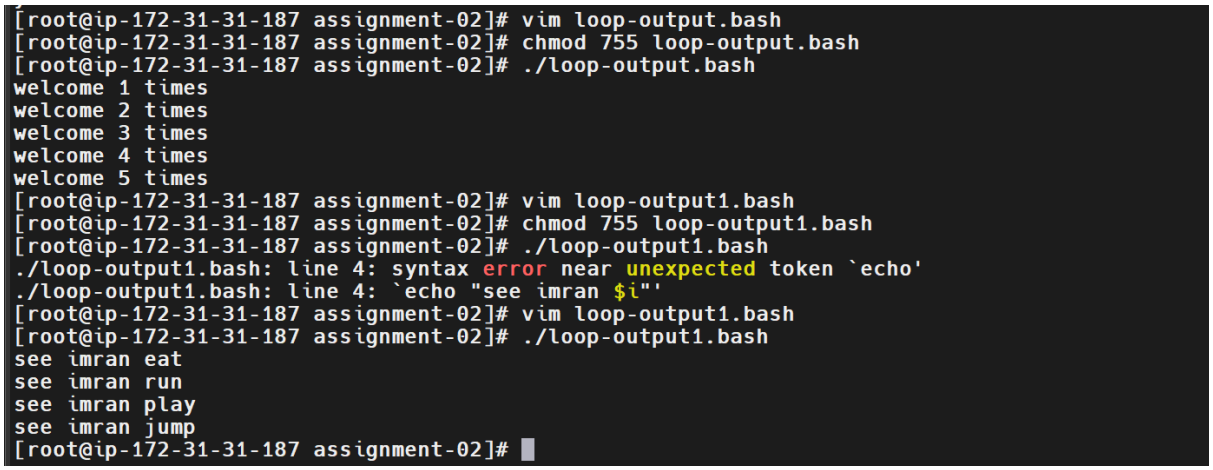
chmod 755 loop-output1.bash

Step 4: Execute the script

./loop-output1.bash

Validation Steps

The initial attempt displayed a syntax error near echo. After correcting the script structure, the screenshot shows all four expected lines.



Explanation

- The for loop processes each word in the list.
- The variable i holds the current word.

- echo prints the message.
- done ends the loop.

Real-Time Use Case

A list-based loop can execute commands for multiple service names, environments, or application components.

Common Errors and Troubleshooting

Error	Solution
Syntax error near echo	Check the placement of do and done.
A word is missing	Verify the list in the for statement.
Unexpected output	Check how the variable is used in the message.

Task 8: Print Weekdays Using a for Loop

Outcome

The script prints five weekday labels with their sequence numbers.

Expected output:

weekday 1 : mon

weekday 2 : tue

weekday 3 : wed

weekday 4 : thu

weekday 5 : fri

Objective

To understand Bash arrays, loops, and counters.

Prerequisites

- Bash interpreter.
- Linux terminal access.

Step-by-Step Implementation

Step 1: Create the script

vim days-loop.bash

Step 2: Add the script

#!/bin/bash

days=(mon tue wed thu fri)

i=1

for day in "${days[@]}"

do

  echo "weekday $i : $day"

  ((i++))

done

Step 3: Provide execute permissions

chmod 755 days-loop.bash

Step 4: Execute the script

./days-loop.bash

```
[root@ip-172-31-31-187 assignment-02]# vim days-loop.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 days-loop.bash
[root@ip-172-31-31-187 assignment-02]# ./days-loop.bash
weekday 1 : mon
weekday 2 : tue
weekday 3 : wed
weekday 4 : thu
weekday 5 : fri
```

Validation Steps

The screenshot shows all five weekdays in sequence, from Monday to Friday.

Explanation

- days=(...) creates an array.
- "${days[@]}" accesses all array elements.
- day stores the current weekday.
- i=1 initializes the counter.
- ((i++)) increments the counter.
- done ends the loop.

Real-Time Use Case

Arrays and loops can be used to process a list of scheduled workdays or perform operations across multiple environments.

Common Errors and Troubleshooting

Error	Solution
Only one array element appears	Use "${days[@]}" to iterate through all elements.

Counter does not increment	Ensure ((i++)) is inside the loop.
Incorrect sequence	Initialize the counter before the loop.

Task 9: Display System Usernames from /etc/passwd

Outcome

The script displays usernames from the Linux account file with a sequence number.

Example output:

Username 1 : root

Username 2 : bin

Username 3 : daemon

Username 4 : adm

Username 5 : lp

The screenshot shows additional system accounts continuing in the same format.

Objective

To understand reading a Linux system file and processing its contents using a loop.

Prerequisites

- Linux system with /etc/passwd.
- Read access to /etc/passwd.
- Bash interpreter.

Step-by-Step Implementation

Step 1: Create the script

vim user-loop.bash

Step 2: Add the script

```
#!/bin/bash


i=1


while IFS=: read -r username _
do
   echo "Username $i : $username"
   ((i++))
done < /etc/passwd
```

Step 3: Provide execute permissions

chmod 755 user-loop.bash

Step 4: Execute the script

./user-loop.bash

```
[root@ip-172-31-31-187 assignment-02]# vim user-loop.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 user-loop.bash
[root@ip-172-31-31-187 assignment-02]# ./user-loop.bash
Username 1 : root
Username 2 : bin
Username 3 : daemon
Username 4 : adm
Username 5 : lp
Username 6 : sync
Username 7 : shutdown
Username 8 : halt
Username 9 : mail
Username 10 : operator
Username 11 : games
Username 12 : ftp
Username 13 : nobody
Username 14 : dbus
Username 15 : systemd-network
Username 16 : systemd-oom
Username 17 : systemd-resolve
Username 18 : rpc
Username 19 : libstoragemgmt
Username 20 : systemd-coredump
Username 21 : systemd-timesync
Username 22 : sshd
Username 23 : chrony
Username 24 : ec2-instance-connect
Username 25 : stapunpriv
Username 26 : rpcuser
```

Validation Steps

Verify that the script displays usernames such as root, bin, and daemon, followed by other accounts listed in /etc/passwd.

Explanation

- /etc/passwd contains local Linux account information.
- while reads the file line by line.
- IFS=: sets the field separator to a colon.
- read -r username _ stores the first field in username.
- i tracks the number of entries processed.
- ((i++)) increments the counter.
- < /etc/passwd redirects the file contents into the loop.

The output includes system and service accounts, not only accounts used by human users.

Real-Time Use Case

Linux administrators can use scripts like this to review local account names during system administration and account audits.

Common Errors and Troubleshooting

Error	Solution
No output appears	Check that /etc/passwd exists and the redirection is correct.

Fields are parsed incorrectly	Use IFS=: with the read command.
Only one line is processed	Check the while loop and the placement of done.

Task 10: Run a Countdown Using a while Loop

Outcome

The script displays a countdown and then prints a final message.

Expected output:

10 seconds left to stop this process

9 seconds left to stop this process

8 seconds left to stop this process

...

2 seconds left to stop this process

1 seconds left to stop this process

process is stopped!!!

Objective

To understand loops, counters, and delays using the sleep command.

Prerequisites

- Bash interpreter.
- Linux terminal access.
- sleep utility available on the system.

Step-by-Step Implementation

Step 1: Create the script

vim do-while.bash

Step 2: Add the script

```
#!/bin/bash


count=10


while [ "$count" -gt 0 ]
do
   echo "$count seconds left to stop this process"
   sleep 1
```

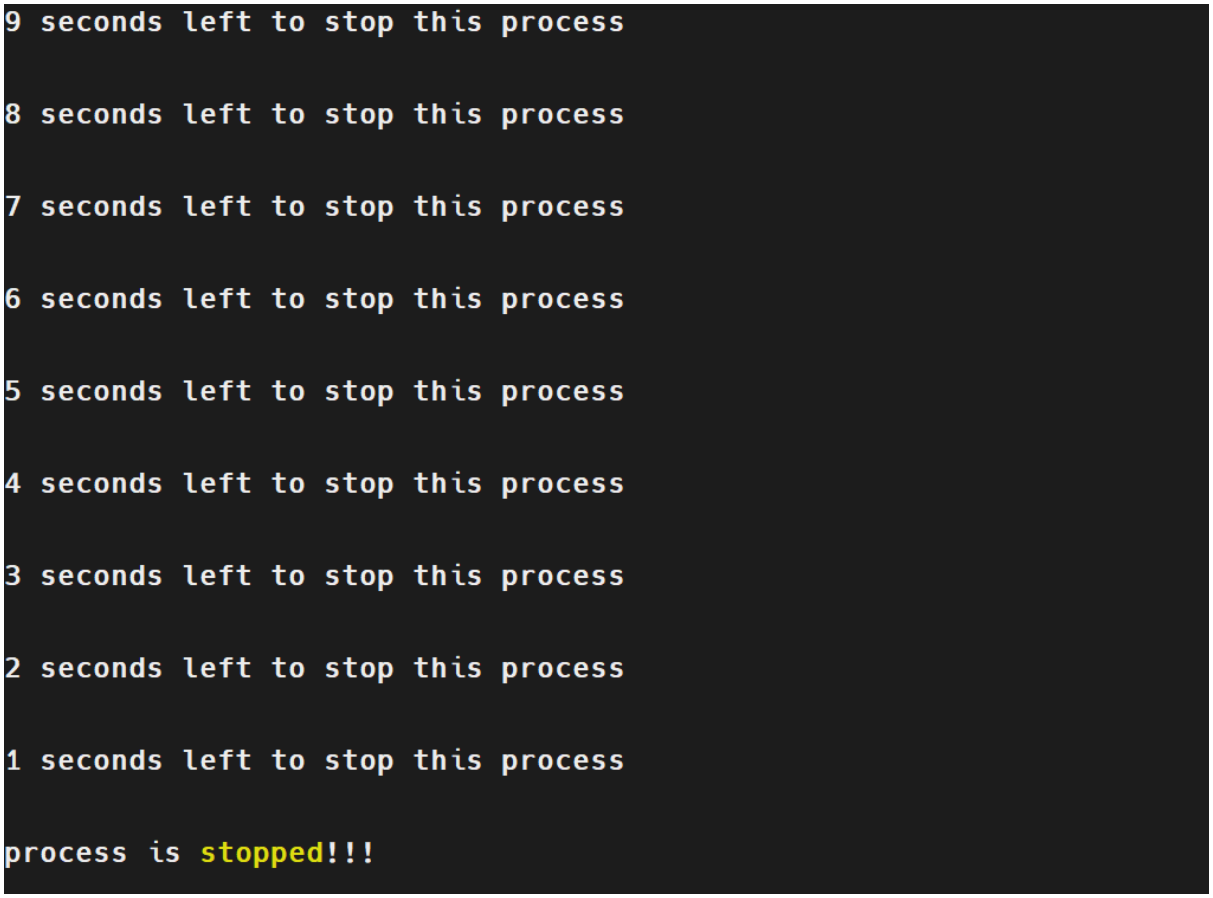
((count--))

done

echo "process is stopped!!!"

Step 3: Execute the script

bash do-while.bash



Validation Steps

The screenshot shows the countdown decreasing once per second and then displays:

process is stopped!!!

Explanation

- count=10 initializes the countdown.
- while ["$count" -gt 0] continues while the counter is greater than zero.
- echo prints the current count.
- sleep 1 pauses execution for approximately one second.
- ((count--)) decreases the counter by one.
- done ends the loop.

- The final echo runs after the loop finishes.

Note: Bash does not have a native do-while keyword. This example uses a while loop to implement a countdown. Also, grammatically, 1 second would be more appropriate than 1 seconds.

Real-Time Use Case

A countdown can introduce a short delay before a controlled operation. It should not replace proper process supervision or safe shutdown procedures.

Common Errors and Troubleshooting

Error	Solution
Countdown does not end	Ensure the counter decreases inside the loop.
Final message does not appear	Check that the final echo is outside the loop.
sleep command fails	Verify that the command is available in the system's PATH.

Overall Validation Steps

1. Verify the working directory:

pwd

Expected directory:

/bash/assignment-02

2. List the scripts and generated files:

ls -l

3. Run each script individually and verify its output.
4. For the file-creation exercise, confirm the files exist:

ls -l 1 2 3 4 5

5. Capture screenshots showing the script execution and successful output.
6. Retain screenshots of initial errors and corrected executions as troubleshooting evidence.
7. Review the script files before pushing them to GitHub.

Conclusion

This document demonstrate Bash scripting practice involving variables, conditional statements, file-existence checks, date commands, user input, loops, arrays, reading Linux account records, and countdowns. Several initial syntax errors were corrected during practice. The screenshots provide evidence of successful execution for the corrected examples.

Task 1: Check Whether a Directory Exists Using a Bash Script

1. Outcome

Successfully created and executed a Bash script named directory.bash on an Amazon Linux EC2 instance to check whether a specified directory exists. Verified both conditions: when the directory exists and when it does not exist.

2. Objective

To understand how to use conditional statements and the -d test operator in Bash scripting to check whether a directory exists. This is useful in DevOps automation to validate directory paths before performing operations such as creating files, storing backups, deploying applications, or processing logs.

3. Prerequisites

- Access to an Amazon Linux EC2 instance.
- Root access or a Linux user with appropriate permissions.
- Access to the terminal through SSH.
- Basic knowledge of Linux commands and Bash scripting.
- The Bash shell installed on the system.

4. Step-by-Step Implementation

Step 1: Open the Terminal and Navigate to the Working Directory

The script was created in the assignment-02 directory on the Amazon Linux EC2 instance.

Command:

cd /path/to/assignment-02

Note: Replace the example path with the actual path of your working directory. In the submitted screenshot, the terminal prompt shows assignment-02.

Screenshot Evidence:

[Paste the screenshot showing the working directory and terminal here.]

Step 2: Create the Bash Script

Create a file named directory.bash using the Vim editor.

Command:

vim directory.bash

Enter the following script:

```
#!/bin/bash


DIR="/home/ec2-user/nov"
```

```
if [ -d "$DIR" ]
then
   echo "directory exists"
else
   echo "directory doesnot exist"
fi
```

Save and exit the editor by pressing Esc, typing :wq, and pressing Enter.

Explanation:

- #!/bin/bash specifies Bash as the script interpreter.
- DIR stores the directory path to check.
- if starts the conditional statement.
- [-d "$DIR"] checks whether the specified path exists and is a directory.
- then executes the command when the condition is true.
- else executes the command when the condition is false.
- fi marks the end of the conditional statement.

Step 3: Provide Execute Permission

Execute the following command to make the script executable.

```
chmod 755 directory.bash
```

The 755 permission allows the file owner to read, write, and execute the script, while group members and others can read and execute it.

Step 4: Execute the Script When the Directory Exists

Run the script:

```
./directory.bash
```

Observed Output:

```
directory exists
```

The output confirms that /home/ec2-user/nov existed when the script was executed.

Explanation: When the directory exists, the -d condition evaluates to true and the then block executes.

Step 5: Execute the Script When the Directory Does Not Exist

Modify the script using Vim:

```
vim directory.bash
```

Change the directory path to a path that does not exist:

DIR="/home/ec2-user/nov"

For this test, ensure that /home/ec2-user/nov does not exist. Your submitted screenshot shows the script printing the negative result for this configured path.

Save the file and execute it again:

./directory.bash

Observed Output:

directory doesnot exist

Explanation: When the specified directory is absent, the -d condition evaluates to false and the else block executes.

Screenshot Evidence:

```
[root@ip-172-31-31-187 assignment-02]# vim directory.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 directory.bash
[root@ip-172-31-31-187 assignment-02]# ./directory.bash
./directory.bash: line 9: unexpected EOF while looking for matching `''
[root@ip-172-31-31-187 assignment-02]# vim directory.bash
[root@ip-172-31-31-187 assignment-02]# ./directory.bash
directory exists
[root@ip-172-31-31-187 assignment-02]# vim directory.bash
[root@ip-172-31-31-187 assignment-02]# ./directory.bash
directory doesnot exist
[root@ip-172-31-31-187 assignment-02]# cat directory.bash
#!/bin/bash

DIR="/home/ec2-user/nov"

if [ -d "$DIR" ]
then
echo directory exists
else
echo directory doesnot exist
fi
[root@ip-172-31-31-187 assignment-02]#
```

5. Validation Steps

Validate the task by confirming that:

- The directory.bash file was created successfully.
- The script contains the correct Bash interpreter declaration.
- The chmod 755 directory.bash command was executed.
- Running ./directory.bash displays directory exists when the configured directory exists.
- Running ./directory.bash displays directory doesnot exist when the configured directory is absent.

Validation Screenshot:

The submitted screenshot shows the script execution, both output messages, and the final script contents. It also shows an initial syntax error that was corrected before successful execution.

6. Explanation

Command or syntax	Description
#!/bin/bash	Specifies Bash as the interpreter.
DIR="..."	Assigns the directory path to a variable.
if	Starts a conditional statement.
-d	Tests whether a path exists and is a directory.
then	Runs commands when the condition is true.
else	Runs commands when the condition is false.
fi	Ends the conditional statement.
echo	Prints a message to the terminal.
chmod 755	Sets read, write, and execute permissions for the owner and read/execute permissions for group and others.
./directory.bash	Executes the script from the current directory.

7. Real-Time Use Case

DevOps engineers use directory checks in deployment and backup scripts to ensure that the required directory is available before performing an operation.

For example, a script can check whether an application directory exists before deploying files:

```
#!/bin/bash

DIR="/var/www/html"

if [ -d "$DIR" ]
then
   echo "Application directory exists"
else
   echo "Application directory does not exist"
fi
```

This helps prevent scripts from proceeding under the assumption that a required directory is available.

8. Common Errors and Troubleshooting

Error or issue	Cause	Resolution
unexpected EOF while looking for matching ...	A quote or other syntax element was left unclosed.	Check the script for missing quotes, brackets, or other closing syntax.
Permission denied	The script does not have execute permission or the user lacks access.	Run chmod 755 directory.bash and verify file permissions.
No such file or directory	The script or configured path does not exist.	Verify the file name and directory path.
Incorrect directory result	The path in DIR does not match the intended directory.	Verify the path using ls -ld /home/ec2-user/nov.
Script does not execute	The script may have syntax errors or an incorrect interpreter declaration.	Review the script and run bash -n directory.bash to check syntax.

Actual error encountered and resolution:

During the initial execution, the script displayed the error:

./directory.bash: line 9: unexpected EOF while looking for matching ...

The script was reopened in Vim and corrected. After the correction, it executed successfully and displayed both the directory-exists and directory-does-not-exist messages during testing.

Conclusion: The task demonstrated how to use Bash conditional statements and the -d operator to check whether a directory exists. This technique is useful for validating paths and making Linux automation scripts more reliable.

Task 2: Create Multiple Files Using a Bash Script

1. Outcome

Successfully created and executed a Bash script named multiple_files.bash on an Amazon Linux EC2 instance to create multiple files automatically. The script created five files (file1.txt to file5.txt) inside /tmp/devops-files using a for loop and the touch command.

2. Objective

To understand how to use Bash scripting, variables, loops, and Linux file-management commands to automate the creation of multiple files. This is useful in DevOps workflows for generating configuration files, log files, test files, and application-specific files.

3. Prerequisites

- Access to an Amazon Linux EC2 instance.
- Root access or a Linux user with appropriate permissions.
- Access to the terminal through SSH.
- Basic knowledge of Bash scripting and Linux commands.
- The Bash shell installed on the system.

4. Step-by-Step Implementation

Step 1: Create the Bash Script

Create a script named multiple_files.bash using Vim.

Command:

vim multiple_files.bash

Enter the following script:

```
#!/bin/bash
clear
DIR="/tmp/devops-files"
mkdir -p "$DIR"
for i in 1 2 3 4 5
do
  touch "$DIR/file$i.txt"
  echo "Created file$i.txt"
done
echo "All files created successfully"
```

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Explanation:

- #!/bin/bash specifies Bash as the script interpreter.
- clear clears the terminal screen.
- DIR="/tmp/devops-files" stores the target directory path in a variable.
- mkdir -p "$DIR" creates the directory if it does not already exist.
- for i in 1 2 3 4 5 iterates through five numbers.
- touch "$DIR/file$i.txt" creates a file for each iteration.
- echo displays a message for each file created.
- done marks the end of the loop.

Step 2: Provide Execute Permission

Execute the following command to make the script executable:

chmod 755 multiple_files.bash

This grants the file owner read, write, and execute permissions, while group members and others receive read and execute permissions.

Screenshot Evidence: Paste the screenshot showing the permission command here.

Step 3: Execute the Script

Run the script from the current directory:

./multiple_files.bash

Observed Output:

Created file1.txt

Created file2.txt

Created file3.txt

Created file4.txt

Created file5.txt

All files created successfully

Explanation: The script iterates through the five values and creates a corresponding file during each iteration. The final message indicates that the loop has finished.

Step 4: Verify the Created Files

Run the following command to list the files:

ls -l /tmp/devops-files

The output should contain these five filenames:

file1.txt

file2.txt

file3.txt

file4.txt

file5.txt

To count the files in the directory, run:

find /tmp/devops-files -maxdepth 1 -type f | wc -l

Expected output:

```
Created file1.txt
Created file2.txt
Created file3.txt
Created file4.txt
Created file5.txt
All files created successfully
[root@ip-172-31-31-187 assignment-02]# cat multiple_files.bash
#!/bin/bash

clear

DIR="/tmp/devops-files"

mkdir -p "$DIR"

for i in 1 2 3 4 5
do
    touch "$DIR/file$i.txt"
    echo "Created file$i.txt"
done

echo "All files created successfully"
[root@ip-172-31-31-187 assignment-02]#
```

Note: The submitted screenshot confirms the creation messages and displays the script contents. The file listing and count are additional validation steps; capture them after executing the commands.

5. Validation Steps

Validate the task by confirming that:

- The multiple_files.bash script was created successfully.
- The script contains the correct Bash interpreter declaration.
- The chmod 755 multiple_files.bash command was executed.
- Running ./multiple_files.bash displays five file creation messages.
- The final message All files created successfully appears.
- The ls -l /tmp/devops-files command lists all five files.
- The find and wc -l commands confirm that five regular files exist in the target directory.

6. Explanation

Command or syntax	Description
#!/bin/bash	Specifies Bash as the script interpreter.
Clear	Clears the terminal screen.
DIR="/tmp/devops-files"	Defines the target directory using a variable.
mkdir -p	Creates the directory if it does not already exist.
for i in 1 2 3 4 5	Repeats the commands for five values.
do	Marks the beginning of the loop body.
touch	Creates empty files if they do not exist.

"$DIR/file$i.txt"	Builds each filename using the directory and loop variable.
echo	Displays messages on the terminal.
done	Ends the loop.
chmod 755	Sets executable permissions on the script.
ls -l	Displays detailed file information.
wc -l	Counts lines; here, it counts the file paths returned by find.

7. Real-Time Use Case

DevOps engineers often automate the creation of multiple files during application setup, testing, and deployment. For example, files for different environments can be created using a loop.

```
#!/bin/bash


for env in dev test prod
do
  touch "${env}.conf"
  echo "Created ${env}.conf"
done
```

This script creates three configuration files:

- dev.conf
- test.conf
- prod.conf

Using loops reduces repetitive manual work and makes scripts easier to maintain.

8. Common Errors and Troubleshooting

Error or issue	Cause	Resolution
Permission denied	Insufficient permissions on the target directory.	Select a writable directory or use appropriate authorized permissions.
No such file or directory	The target path is incorrect or missing.	Check the path and use mkdir -p to create the directory.
./multiple_files.bash: Permission denied	Execute permission is missing.	Run chmod 755 multiple_files.bash.
Files are not visible	The wrong directory is being checked.	Run ls -l /tmp/devops-files.

Only some files are created	The loop or command inside it may be incorrect.	Check the loop syntax and execute the script again.
Script runs but files are empty	touch creates empty files by design.	Use echo or another command to write content if required.

Actual error encountered / resolution: No execution errors were visible in the submitted screenshot. The script displayed all five creation messages and the final success message.

Conclusion: The task demonstrated how to use Bash loops, variables, mkdir, and touch to automate file creation. This technique is useful for repetitive file-management operations in Linux administration and DevOps automation.

Task 3: Take a Backup of a Directory Using a Bash Script

1. Outcome

Successfully created and executed a Bash script named backup_directory.bash on an Amazon Linux EC2 instance to create a compressed backup of a directory. Verified the backup file and practiced extracting the archive into a separate directory.

2. Objective

To understand how to automate directory backups using Bash scripting and the tar command. This is useful in DevOps for protecting application files, configuration files, and other important data before deployments, maintenance, or system changes.

3. Prerequisites

- Access to an Amazon Linux EC2 instance.
- Root access or appropriate directory permissions.
- Basic knowledge of Bash scripting and Linux commands.
- The tar utility installed on the system.
- Sufficient disk space to store the backup.

4. Step-by-Step Implementation

Step 1: Create a Sample Directory and Files

Create a directory to store sample application files.

Commands:

mkdir -p /tmp/devops-backup-source

echo "Application configuration" > /tmp/devops-backup-source/app.conf

echo "Backup practice file" > /tmp/devops-backup-source/readme.txt

Verify the files:

ls -l /tmp/devops-backup-source

Expected files:

app.conf

readme.txt

Step 2: Create the Bash Script

Create a file named backup_directory.bash.

Command:

vim backup_directory.bash

Enter the following script:

```
#!/bin/bash

SOURCE_DIR="/tmp/devops-backup-source"
BACKUP_DIR="/tmp/devops-backups"
DATE=$(date +%Y%m%d_%H%M%S)

if [ ! -d "$SOURCE_DIR" ]; then
   echo "Source directory does not exist: $SOURCE_DIR"
   exit 1
fi

mkdir -p "$BACKUP_DIR"

BACKUP_FILE="$BACKUP_DIR/backup_$DATE.tar.gz"

if tar -czf "$BACKUP_FILE" -C "$SOURCE_DIR" .; then
   echo "Backup created successfully: $BACKUP_FILE"
else
   echo "Backup failed"
   exit 1
fi

if [ -s "$BACKUP_FILE" ]; then
```

```
    echo "Backup verification successful"
    ls -lh "$BACKUP_FILE"
else
    echo "Backup file is missing or empty"
    exit 1
fi
```

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Explanation:

- SOURCE_DIR specifies the directory to back up.
- BACKUP_DIR specifies the directory where the backup will be stored.
- date generates a timestamp for the backup filename.
- if [! -d "$SOURCE_DIR"] checks whether the source directory is missing.
- mkdir -p creates the backup directory if it does not exist.
- tar -czf creates a compressed archive.
- if [-s "$BACKUP_FILE"] checks whether the backup file exists and is non-empty.

Step 3: Provide Execute Permission

Command:

```
chmod 755 backup_directory.bash
```

This makes the script executable by the owner, group, and other users, subject to their permissions.

Step 4: Execute the Script

Command:

```
./backup_directory.bash
```

Example output:

```
Backup created successfully: /tmp/devops-backups/backup_20261025_085522.tar.gz
Backup verification successful
```

The timestamp in the filename depends on the actual execution time. Your script should display the actual filename it creates.

Step 5: Verify the Backup File

Command:

```
ls -lh /tmp/devops-backups/
```

This displays the backup archive and its size.

To inspect the archive contents, use:

tar -tzf /tmp/devops-backups/backup_20261025_085522.tar.gz

Replace the example filename with the actual filename printed by your script if it differs.

Expected entries include:

./

./app.conf

./readme.txt

Step 6: Create a Restore Directory

Command:

mkdir -p /tmp/devops-backup-restore

This creates a separate directory for testing the backup extraction.

Step 7: Extract the Backup

Use the full path to the backup archive. Execute the entire command on one line.

Command:

tar -xzf /tmp/devops-backups/backup_20261025_085522.tar.gz -C /tmp/devops-backup-restore

Replace the example filename with the actual archive filename if necessary.

Explanation:

- -x extracts files from an archive.
- -z handles gzip compression.
- -f specifies the archive filename.
- -C changes to the specified destination directory before extracting.

During testing, an initial extraction attempt failed because the archive filename was not found in the current directory and the -C option was entered on a separate line. The command was corrected by using the full archive path and keeping all options in the same command.

Step 8: Verify the Restored Files

Commands:

ls -l /tmp/devops-backup-restore

cat /tmp/devops-backup-restore/app.conf

Expected content of app.conf:

Application configuration

The restored files confirm that the archive can be extracted and its contents accessed.

5. Validation Steps

Validate the task by confirming that:

- The sample source directory was created successfully.
- The backup_directory.bash script was created and made executable.
- The script checks whether the source directory exists.
- The tar command creates a compressed backup with a timestamp.
- The backup file exists and is non-empty.
- The tar -tzf command displays the archived files.
- The backup is extracted into /tmp/devops-backup-restore.
- The restored app.conf file contains Application configuration.

Screenshot Evidence:

```
[root@ip-172-31-31-187 assignment-02]# mkdir -p /tmp/devops-backup-source
echo "Application configuration" > /tmp/devops-backup-source/app.conf
echo "Backup practice file" > /tmp/devops-backup-source/readme.txt
[root@ip-172-31-31-187 assignment-02]# ls -l /tmp/devops-backup-source
total 8
-rw-r--r--. 1 root root 26 Oct 25 08:51 app.conf
-rw-r--r--. 1 root root 21 Oct 25 08:51 readme.txt
[root@ip-172-31-31-187 assignment-02]# vim backup_directory.bash
```

```
[root@ip-172-31-31-187 assignment-02]# chmod 755 backup-directory.bash
chmod: cannot access 'backup-directory.bash': No such file or directory
[root@ip-172-31-31-187 assignment-02]# chmod 755 backup_directory.bash
[root@ip-172-31-31-187 assignment-02]# ./backup_directory.bash
Backup created successfully: /tmp/devops-backups/backup_20261025_085522.tar.gz
Backup verification successful
-rw-r--r--. 1 root root 213 Oct 25 08:55 /tmp/devops-backups/backup_20261025_085522.tar.gz
```

```
[root@ip-172-31-31-187 assignment-02]# cat backup_directory.bash
#!/bin/bash

SOURCE_DIR="/tmp/devops-backup-source"
BACKUP_DIR="/tmp/devops-backups"
DATE=$(date +%Y%m%d_%H%M%S)

if [ ! -d "$SOURCE_DIR" ]; then
    echo "Source directory does not exist: $SOURCE_DIR"
    exit 1
fi

mkdir -p "$BACKUP_DIR"

BACKUP_FILE="$BACKUP_DIR/backup_$DATE.tar.gz"

if tar -czf "$BACKUP_FILE" -C "$SOURCE_DIR" .; then
    echo "Backup created successfully: $BACKUP_FILE"
else
    echo "Backup failed"
    exit 1
fi

if [ -s "$BACKUP_FILE" ]; then
    echo "Backup verification successful"
    ls -lh "$BACKUP_FILE"
else
    echo "Backup file is missing or empty"
    exit 1
fi
```

```
[root@ip-172-31-31-187 assignment-02]#
[root@ip-172-31-31-187 assignment-02]# ls -lh /tmp/devops-backups
total 4.0K
-rw-r--r--. 1 root root 213 Oct 25 08:55 backup_20261025_085522.tar.gz
[root@ip-172-31-31-187 assignment-02]#
```

```
[root@ip-172-31-31-187 assignment-02]# tar -xzf /tmp/devops-backups/backup_20261025_085522.tar.gz -C /tmp/devops-backup-res
tore
[root@ip-172-31-31-187 assignment-02]# ls -l /tmp/devops-backup-restore
total 8
-rw-r--r--. 1 root root 26 Oct 25 08:51 app.conf
-rw-r--r--. 1 root root 21 Oct 25 08:51 readme.txt
[root@ip-172-31-31-187 assignment-02]# ls -l /tmp/devops-backup-restore
total 8
-rw-r--r--. 1 root root 26 Oct 25 08:51 app.conf
-rw-r--r--. 1 root root 21 Oct 25 08:51 readme.txt
```

6. Explanation

Command or option	Description
mkdir -p	Creates a directory and any missing parent directories.
date +%Y%m%d_%H%M%S	Generates a timestamp for a unique filename.
tar -czf	Creates a gzip-compressed archive.
-C	Specifies the directory in which to operate during archive creation or extraction.
tar -tzf	Lists the contents of a compressed archive.
tar -xzf	Extracts a gzip-compressed archive.
[! -d "$SOURCE_DIR"]	Checks whether the source directory does not exist.
[-s "$BACKUP_FILE"]	Checks whether the file exists and has a size greater than zero.
ls -lh	Displays file details and human-readable sizes.
exit 1	Exits the script with a non-zero status to indicate failure.

7. Real-Time Use Case

DevOps engineers can create backups before deploying a new application version or modifying configuration files. If a deployment fails, the backup can help restore the previous files.

For example, an automated backup can be created before a maintenance operation:

tar -czf app_backup_$(date +%Y%m%d_%H%M%S).tar.gz /var/www/html

This command archives the application directory into a timestamped compressed file.

Important: A backup stored only on the same EC2 instance does not protect against instance or disk loss. For production environments, consider storing backups in Amazon S3, applying retention policies, and testing restoration regularly.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution

Source directory does not exist	Incorrect source path	Verify the path using ls -ld.
Cannot open: No such file or directory	The archive filename or path is incorrect	Use the full archive path and verify it with ls -lh.
bash: -C: command not found	The -C option was entered as a separate command	Keep the complete tar command on one line.
Permission denied	Insufficient directory permissions	Check permissions and use authorized privileges.
No space left on device	Insufficient disk space	Check filesystem usage using df -h.
Backup file is empty	Archive creation failed or source data was unavailable	Check the tar command's exit status and inspect the source directory.
Cannot extract archive	Incorrect filename or damaged archive	Verify the filename and test the archive with tar -tzf.

Actual error encountered and resolution:

The initial extraction command failed because the backup archive was not found in the current working directory. The -C option was also entered on a separate line, causing Bash to interpret it as a command. The resolution was to use the full archive path and execute the complete tar -xzf ... -C ... command on one line.

Conclusion: This task demonstrated how to automate directory backups using Bash scripting and tar, verify compressed archives, and restore files into a separate directory. These practices support safer deployments, recovery procedures, and routine DevOps maintenance.

Task 4: Install Nginx on an Amazon Linux EC2 Instance Using a Bash Script

1. Outcome

Successfully created and executed a Bash script named install_nginx.bash on an Amazon Linux EC2 instance to automate the installation of Nginx. Verified that the Nginx service was started, enabled to start automatically at boot, and responding to HTTP requests locally.

2. Objective

To understand how to automate software installation and service management using Bash scripting on an Amazon Linux EC2 instance. Nginx is a web server commonly used to serve web applications, reverse-proxy requests, and distribute traffic to backend services.

3. Prerequisites

- Access to an Amazon Linux EC2 instance.
- Root access or a user with appropriate sudo privileges.
- SSH or terminal access to the instance.
- Internet connectivity to access configured package repositories.

- The dnf or yum package manager.
- An EC2 security group configured appropriately if external HTTP access is required.

4. Step-by-Step Implementation

Step 1: Verify the Operating System

Run the following command to confirm the Linux distribution:

cat /etc/os-release

This confirms the operating system and helps determine the appropriate package manager.

Step 2: Create the Bash Script

Create a script named install_nginx.bash:

vim install_nginx.bash

Enter the following script:

```
#!/bin/bash

set -e

echo "Installing Nginx..."

if command -v dnf >/dev/null 2>&1; then
   dnf install -y nginx
elif command -v yum >/dev/null 2>&1; then
   yum install -y nginx
else
   echo "No supported package manager found"
   exit 1
fi

systemctl enable --now nginx

echo "Checking Nginx service status..."
systemctl status nginx --no-pager
```

echo "Nginx installation completed successfully"

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Explanation:

- #!/bin/bash specifies Bash as the script interpreter.
- set -e causes the script to exit if a command fails, subject to Bash's set -e behavior.
- command -v dnf checks whether the dnf package manager is available.
- dnf install -y nginx installs Nginx without interactive confirmation.
- yum install -y nginx provides an alternative for systems using yum.
- systemctl enable --now nginx enables Nginx at boot and starts it immediately.
- systemctl status nginx --no-pager displays the service status without opening a pager.

Step 3: Provide Execute Permission

Run:

chmod 755 install_nginx.bash

This makes the script executable according to the configured permissions.

Step 4: Execute the Script

Run:

./install_nginx.bash

The script installs Nginx, starts the service, enables it at boot, and displays its status.

Expected result: The output indicates that Nginx has been installed and that the service is active.

Step 5: Verify the Nginx Service

Check whether the service is running:

systemctl is-active nginx

Expected output:

active

Check whether it is enabled at boot:

systemctl is-enabled nginx

Expected output:

enabled

Step 6: Verify the HTTP Response

Run:

curl -I http://localhost

A typical successful response begins with:

HTTP/1.1 200 OK

The exact response headers may differ. A successful HTTP response confirms that the local web server is responding.

Step 7: Access Nginx from a Browser (Optional)

To access Nginx using your EC2 public IPv4 address:

1. Open the AWS Management Console.
2. Navigate to EC2 → Instances.
3. Select your instance and copy its public IPv4 address.
4. Open the instance's associated security group.
5. Add an inbound rule for HTTP traffic on TCP port 80, restricted to an appropriate source.
6. Open http://YOUR_EC2_PUBLIC_IP in your browser.

Do not open port 80 to the entire internet unless that access is appropriate for your environment.

5. Validation Steps and Screenshot

Validate the task by confirming that:

- The install_nginx.bash script was created successfully.
- Execute permissions were applied using chmod 755.
- The script installed Nginx using the available package manager.
- systemctl is-active nginx returns active.
- systemctl is-enabled nginx returns enabled.
- curl -I http://localhost returns a successful HTTP response.
- If external browser access was tested, the Nginx welcome page loads successfully.

Screenshot Evidence:

```
Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/s
Checking Nginx service status...
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Sun 2026-10-25 09:21:18 UTC; 16ms ago
    Process: 5831 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Process: 5868 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
    Process: 5921 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 5998 (nginx)
```

```
[root@ip-172-31-31-187 assignment-02]# cat install_nginx.bash
#!/bin/bash

set -e

echo "Installing Nginx..."

if command -v dnf >/dev/null 2>&1; then
    dnf install -y nginx
elif command -v yum >/dev/null 2>&1; then
    yum install -y nginx
else
    echo "No supported package manager found"
    exit 1
fi

systemctl enable --now nginx

echo "Checking Nginx service status..."
systemctl status nginx --no-pager

echo "Nginx installation completed successfully"
```

```
[root@ip-172-31-31-187 assignment-02]# curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.30.5
Date: Sun, 25 Oct 2026 09:24:59 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Wed, 16 Sep 2026 21:09:29 GMT
Connection: keep-alive
ETag: "6aab0589-380"
Accept-Ranges: bytes
```

6. Explanation

Command or option	Description
#!/bin/bash	Specifies Bash as the script interpreter.
set -e	Exits the script when an eligible command fails.
command -v dnf	Checks whether dnf is available.
dnf install -y nginx	Installs Nginx using dnf.
yum install -y nginx	Installs Nginx using yum.
systemctl enable --now nginx	Enables Nginx at boot and starts it immediately.
systemctl status nginx	Displays detailed service status.
systemctl is-active nginx	Checks whether Nginx is active.
systemctl is-enabled nginx	Checks whether Nginx is enabled at boot.

curl -I http://localhost	Sends an HTTP request and displays response headers.
chmod 755	Sets the script's file permissions.

7. Real-Time Use Case

DevOps engineers use Bash scripts to automate web-server installation during server provisioning and application deployment.

For example, Nginx may serve static website content or act as a reverse proxy that forwards incoming HTTP requests to backend application services.

Automating installation and service startup helps standardize server configuration and reduces repetitive manual work.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
dnf: command not found	The package manager is unavailable.	Check whether yum is available and verify the OS.
Unable to download packages	Repository or network connectivity issue.	Check network access and configured package repositories.
Permission denied	Insufficient privileges.	Run the script as root or with authorized sudo privileges.
Unit nginx.service could not be found	Nginx is not installed or the installation failed.	Check the installation output and verify with rpm -q nginx.
Nginx service fails to start	Configuration error or port conflict.	Inspect systemctl status nginx and journalctl -u nginx.
curl: (7) Failed to connect	Nginx is stopped or not listening on the expected port.	Check systemctl status nginx and ss -lntp.
Local access works but external access fails	Security group, network ACL, or routing restriction.	Verify inbound TCP port 80 and network configuration.

Actual error encountered / resolution: Record any errors that occurred during your EC2 installation and the steps you used to resolve them. If no errors occurred, document that the installation completed successfully without errors.

Conclusion: This task demonstrated how to automate Nginx installation on Amazon Linux using Bash scripting, manage its service with systemctl, and validate the web server using curl. These skills are useful for automated server provisioning and DevOps deployment workflows.

Task 5: Install Apache Tomcat on Amazon Linux EC2 Using a Bash Script

Date: 20261025

1. Outcome

Successfully installed and configured Apache Tomcat on an Amazon Linux 2023 EC2 instance using a Bash script. Installed Java 17 (Amazon Corretto), configured Tomcat as a systemd service, and verified that Tomcat is working.

2. Objective

To automate the installation and configuration of Apache Tomcat on an Amazon Linux EC2 instance using Bash scripting.

The script performs the following operations:

- Checks and installs required utilities.
- Verifies the Java installation.
- Creates a dedicated Tomcat system user.
- Downloads and extracts the Apache Tomcat archive.
- Configures permissions and ownership.
- Creates a systemd service.
- Starts and enables Tomcat.
- Verifies the service and HTTP response.

3. Prerequisites

- An active AWS account.
- A running Amazon Linux 2023 EC2 instance.
- SSH access to the EC2 instance.
- Root privileges or sudo access.
- Java 17 installed.
- Internet connectivity to download Apache Tomcat.
- Security group access configured if Tomcat needs to be accessed externally.

4. Step-by-Step Implementation

Step 1: Verify the operating system

Command:

cat /etc/os-release

The instance was running Amazon Linux 2023.

Step 2: Install Java

Java was installed separately after encountering a package conflict during the initial script execution.

Command:

java -version

Observed output confirmed Java 17:

openjdk version "17.0.20.1" 2026-08-18 LTS

OpenJDK Runtime Environment Corretto-17.0.20.13.1

64-Bit Server VM

Explanation: Apache Tomcat requires a compatible Java runtime to execute Java-based web applications.

Step 3: Create the Bash script

Command:

vim install_tomcat.bash

Script content:

```
#!/bin/bash

set -e

TOMCAT_VERSION="10.1.52"
TOMCAT_USER="tomcat"
INSTALL_DIR="/opt/tomcat"

echo "Step 1: Checking required utilities..."

if ! command -v curl >/dev/null 2>&1 || \
   ! command -v tar >/dev/null 2>&1; then
    dnf install -y curl tar
fi

echo "Java version:"
java -version

echo "Step 2: Creating Tomcat user..."
```

```
if ! id "$TOMCAT_USER" >/dev/null 2>&1; then
    useradd --system --home-dir "$INSTALL_DIR" \
        --shell /sbin/nologin "$TOMCAT_USER"
fi

echo "Step 3: Downloading Apache Tomcat..."

TMP_DIR=$(mktemp -d)
trap 'rm -rf "$TMP_DIR"' EXIT

curl -fL \
    "https://archive.apache.org/dist/tomcat/tomcat-10/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz" \
    -o "$TMP_DIR/tomcat.tar.gz"

echo "Step 4: Installing Tomcat..."

mkdir -p "$INSTALL_DIR"

tar -xzf "$TMP_DIR/tomcat.tar.gz" \
    --strip-components=1 -C "$INSTALL_DIR"

chown -R "$TOMCAT_USER":"$TOMCAT_USER" "$INSTALL_DIR"
chmod +x "$INSTALL_DIR/bin/"*.sh

echo "Step 5: Creating systemd service..."

cat > /etc/systemd/system/tomcat.service <<EOF
[Unit]
Description=Apache Tomcat Web Application Server
After=network.target
```

```
[Service]
Type=forking
User=$TOMCAT_USER
Group=$TOMCAT_USER
Environment="JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto"
Environment="CATALINA_HOME=$INSTALL_DIR"
Environment="CATALINA_BASE=$INSTALL_DIR"
ExecStart=$INSTALL_DIR/bin/startup.sh
ExecStop=$INSTALL_DIR/bin/shutdown.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF

echo "Step 6: Starting Tomcat..."

systemctl daemon-reload
systemctl enable --now tomcat

echo "Step 7: Checking Tomcat service..."

systemctl --no-pager --full status tomcat

echo "Step 8: Testing Tomcat locally..."

curl -I http://localhost:8080

echo "Tomcat installation completed successfully!"
```

Note: The script uses Tomcat version 10.1.52 as configured for this task. For a new deployment, use a currently supported, patched release and verify the JAVA_HOME path on the instance.

Step 4: Save and grant execution permission

Save the script in Vim using Esc, then :wq, and press Enter.

Grant execution permission:

chmod 755 install_tomcat.bash

Explanation: The chmod command makes the script executable by the owner and readable/executable by other users.

Step 5: Execute the script

Command:

./install_tomcat.bash

The script checks the required utilities, verifies Java, creates the Tomcat user, downloads and extracts Tomcat, configures the service, and starts it.

Step 6: Verify the Tomcat service

Command:

systemctl status tomcat --no-pager

Check whether the service is active:

systemctl is-active tomcat

Expected output:

active

Check whether the service is enabled at startup:

systemctl is-enabled tomcat

Expected output:

enabled

Step 7: Verify HTTP connectivity

Command:

curl -I http://localhost:8080

Expected result:

HTTP/1.1 200

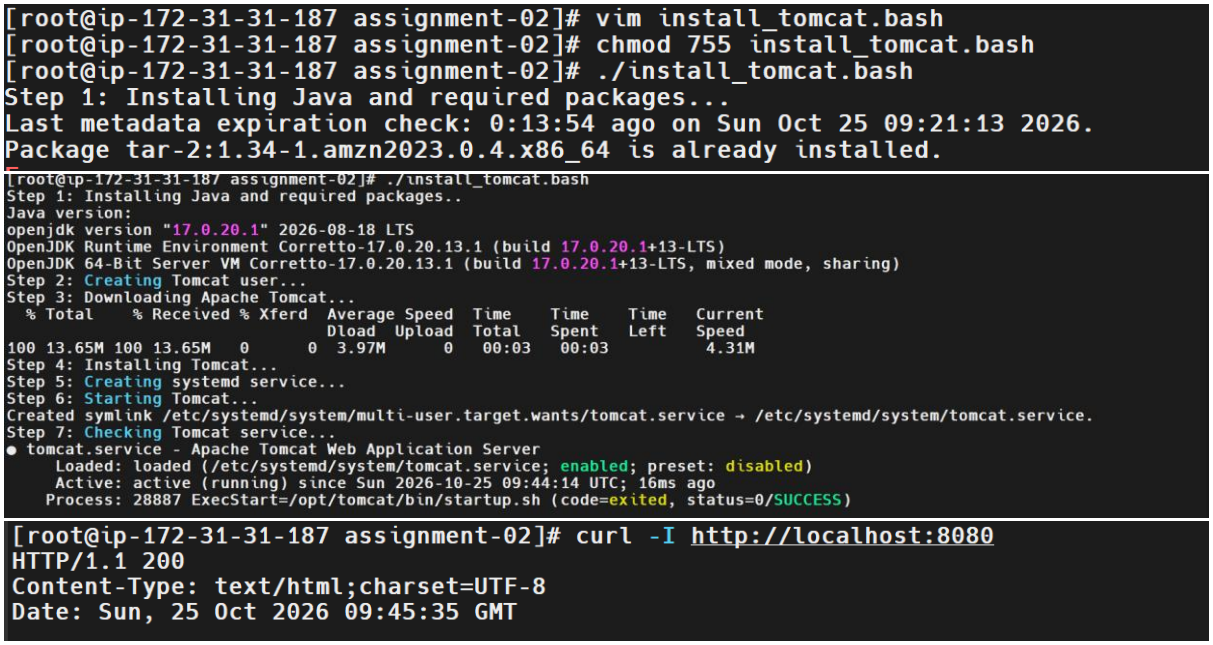
A successful HTTP response confirms that Tomcat is responding locally on port 8080. The exact response headers can vary by Tomcat version and configuration.

5. Validation Steps and Screenshot Evidence

Use the following commands to validate the installation.

Validation	Command	Expected result
Java version	java -version	Java 17 displayed
Tomcat service	systemctl status tomcat	Service running
Active state	systemctl is-active tomcat	active
Startup configuration	systemctl is-enabled tomcat	enabled
HTTP response	curl -I http://localhost:8080	Successful HTTP response
Installation directory	ls -l /opt/tomcat	Tomcat files and directories listed

Screenshot evidence to include in your documentation:



6. Explanation of Commands

Command or option	Description
#!/bin/bash	Specifies Bash as the script interpreter.
set -e	Stops the script when a command fails.
command -v	Checks whether a command is available.
dnf install -y	Installs packages without an interactive confirmation prompt.
java -version	Displays the installed Java version.
id tomcat	Checks whether the Tomcat user exists.

useradd --system	Creates a system user.
mktemp -d	Creates a temporary directory.
curl -fL	Downloads a file, follows redirects, and fails on HTTP error responses.
tar -xzf	Extracts a gzip-compressed tar archive.
--strip-components=1	Removes the top-level directory from archive paths during extraction.
chown -R	Changes ownership recursively.
chmod +x	Adds executable permission.
cat > file <<EOF	Writes a multi-line configuration file using a here-document.
systemctl daemon-reload	Reloads systemd service definitions.
systemctl enable --now	Enables a service at startup and starts it immediately.
curl -I	Requests HTTP headers to test connectivity.

7. Real-Time Use Case

Apache Tomcat is used to host Java web applications in development, testing, and production environments.

For example, a DevOps engineer can automate Tomcat installation on EC2 instances using Bash scripting. This reduces repetitive manual work and helps standardize application server configuration across environments.

The script can be incorporated into server provisioning workflows alongside configuration management and deployment automation tools.

8. Common Errors and Troubleshooting

Error	**Possible cause**	**Troubleshooting**
Java package conflict	Conflicting package dependencies	Install Java separately and inspect the DNF error before changing packages.
Tomcat archive download fails	Invalid or unavailable version URL	Verify the Tomcat version and official download archive.
curl: command not found	Curl is not installed	Install it using dnf install -y curl.
Permission denied	Insufficient privileges or incorrect file permissions	Use an authorized account and check script permissions.
Tomcat service fails	Incorrect Java path, permissions, or service configuration	Run journalctl -u tomcat -n 50 --no-pager.

Port 8080 is unavailable	Another process is using the port	Run ss -lntp | grep ':8080'.
Browser cannot reach Tomcat	Security group or network configuration issue	Check the EC2 security group, public IP, and network connectivity.

Conclusion

Apache Tomcat was installed and configured on Amazon Linux 2023 using a Bash script. Java 17 was installed separately to resolve the initial package conflict. The script automates Tomcat setup, creates a dedicated service user, configures systemd, and starts the service.

The successful service and HTTP checks confirm that the Tomcat server is working. This task demonstrates how Bash scripting can automate application server installation and service management on AWS EC2.

Task 6: Check Nginx Service Status and Start It If Stopped

Outcome

A Bash script named check_nginx.bash was created to check whether the Nginx service is running. If the service is stopped, the script attempts to start it and verifies its final status.

Nginx service status checked successfully.

If Nginx was stopped, the script attempted to start it.

The final service status was verified.

Objective

To automate Nginx service monitoring and basic recovery using a Bash script. This helps DevOps engineers identify a stopped service and attempt to restore it without manually executing service management commands.

Prerequisites

- An AWS account and a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.
- Root privileges or sudo permissions.
- Nginx installed on the EC2 instance.
- Basic knowledge of Bash scripting and Linux service management.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux EC2 instance through SSH or your preferred SSH client.

Verify the operating system:

cat /etc/os-release

Check whether Nginx is installed:

nginx -v

If Nginx is not installed, execute:

dnf install -y nginx

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# nginx -v
nginx version: nginx/1.30.5
```

Step 2: Create the Bash Script

Create a file named check_nginx.bash:

vim check_nginx.bash

Add the following script:

```
#!/bin/bash

SERVICE="nginx"

echo "Checking Nginx service status..."

if ! systemctl list-unit-files nginx.service --no-legend | grep -q nginx; then
    echo "Nginx service is not installed."
    exit 1
fi

if systemctl is-active --quiet "$SERVICE"; then
    echo "Nginx is already running."
else
    echo "Nginx is not running. Starting Nginx..."

    if systemctl start "$SERVICE"; then
        echo "Nginx start command executed."
    else
        echo "Failed to start Nginx."
        systemctl status "$SERVICE" --no-pager
```

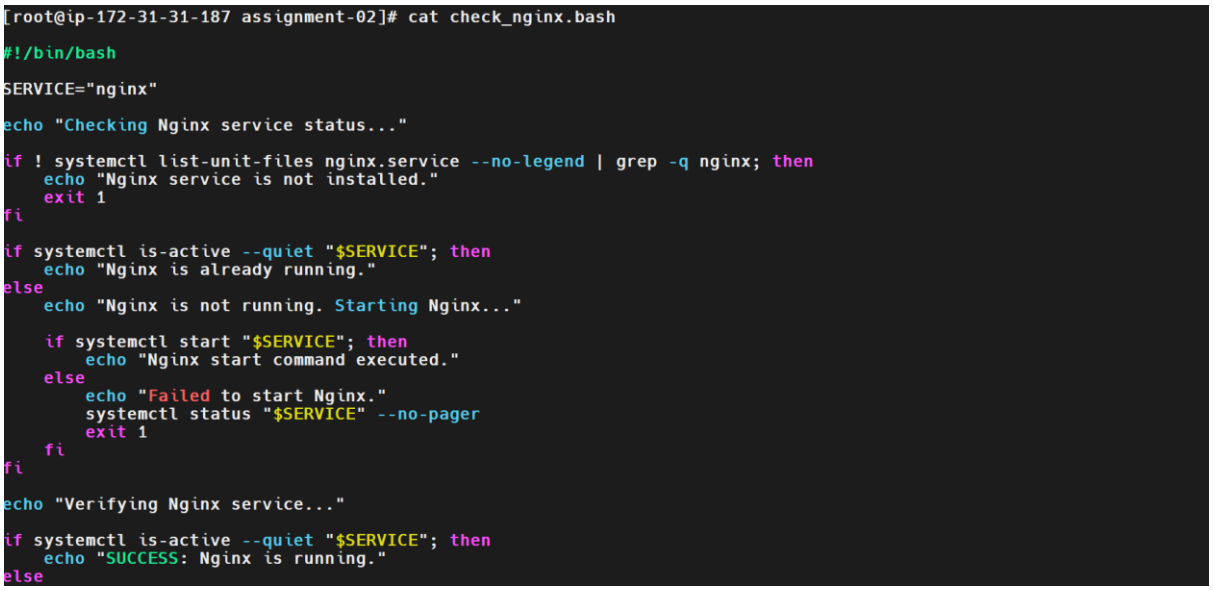
```
        exit 1
    fi
fi

echo "Verifying Nginx service..."

if systemctl is-active --quiet "$SERVICE"; then
    echo "SUCCESS: Nginx is running."
else
    echo "ERROR: Nginx is not running."
    systemctl status "$SERVICE" --no-pager
    exit 1
fi
```

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Screenshot Evidence



```
[root@ip-172-31-31-187 assignment-02]# cat check_nginx.bash
#!/bin/bash

SERVICE="nginx"

echo "Checking Nginx service status..."

if ! systemctl list-unit-files nginx.service --no-legend | grep -q nginx; then
    echo "Nginx service is not installed."
    exit 1
fi

if systemctl is-active --quiet "$SERVICE"; then
    echo "Nginx is already running."
else
    echo "Nginx is not running. Starting Nginx..."

    if systemctl start "$SERVICE"; then
        echo "Nginx start command executed."
    else
        echo "Failed to start Nginx."
        systemctl status "$SERVICE" --no-pager
        exit 1
    fi
fi

echo "Verifying Nginx service..."

if systemctl is-active --quiet "$SERVICE"; then
    echo "SUCCESS: Nginx is running."
else
```

Step 3: Grant Execution Permission

Execute:

chmod 755 check_nginx.bash

This command makes the script executable.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# vim check_nginx.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 check_nginx.bash
[root@ip-172-31-31-187 assignment-02]# ./check_nginx.bash
Checking Nginx service status...
Nginx is already running.
Verifying Nginx service...
SUCCESS: Nginx is running.
```

Step 4: Execute the Script

Run:

./check_nginx.bash

When Nginx is already running, the expected output is:

Checking Nginx service status...

Nginx is already running.

Verifying Nginx service...

SUCCESS: Nginx is running.

When Nginx is stopped, the script attempts to start it and checks the final status.

Screenshot Evidence

Step 5: Test the Stopped-Service Scenario

For testing purposes, stop Nginx:

systemctl stop nginx

Run the script again:

./check_nginx.bash

The script should detect that Nginx is stopped, attempt to start it, and verify whether it is running.

Note: Stopping Nginx interrupts websites served by the service. Perform this test only in your practice environment.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# systemctl stop nginx
[root@ip-172-31-31-187 assignment-02]# ./check_nginx.bash
Checking Nginx service status...
Nginx is not running. Starting Nginx...
Nginx start command executed.
Verifying Nginx service...
SUCCESS: Nginx is running.
```

Validation Steps

Step 1: Check the Service Status

Execute:

systemctl status nginx --no-pager

The output should show that the service is active and running.

Step 2: Verify the Active State

Execute:

systemctl is-active nginx

Expected output:

active

Step 3: Test the HTTP Response

Execute:

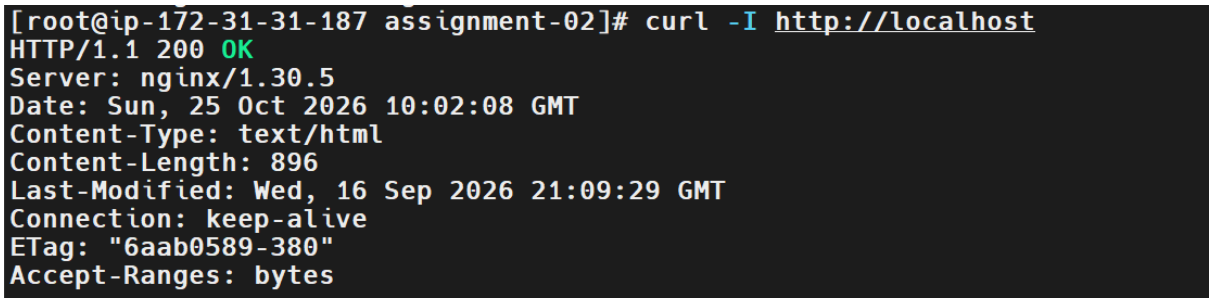
curl -I http://localhost

A successful response commonly begins with:

HTTP/1.1 200 OK

The exact response can vary depending on the server configuration.

Validation Screenshot



Explanation

The script uses systemctl to check and manage the Nginx service. The systemctl is-active --quiet command checks whether Nginx is running without displaying its status directly. If the service is inactive, the script attempts to start it using systemctl start nginx.

After attempting to start the service, the script checks the status again. If Nginx is still inactive, it displays diagnostic information and exits with an error status. This approach helps automate basic service recovery while providing useful troubleshooting information when the service cannot start.

Real-Time Use Case

In real-world DevOps environments, Nginx is commonly used as a web server and reverse proxy. If the service stops unexpectedly, a recovery script can attempt to restart it and verify its status.

Such scripts can be integrated into administrative workflows and monitoring systems. For production environments, monitoring and alerting should also be configured to detect repeated failures and notify the responsible team.

Common Errors and Troubleshooting

Error	Cause	Resolution

Unit nginx.service could not be found	Nginx is not installed or its service unit is unavailable.	Install Nginx using dnf install -y nginx.
Permission denied	The current user lacks permission to manage services.	Run the script as root or use an authorized account with sudo.
Nginx fails to start	Configuration errors or missing dependencies.	Run nginx -t to validate the configuration.
Port 80 is already in use	Another process is listening on port 80.	Run ss -lntp | grep ':80' to identify the process.
HTTP request fails	Nginx is stopped, misconfigured, or not listening correctly.	Check systemctl status nginx and run curl -I http://localhost.
Website is inaccessible externally	Security group or network configuration prevents access.	Check the EC2 security group's inbound rules and network connectivity.

Conclusion

The check_nginx.bash script automates checking the Nginx service status, attempting to start the service when it is stopped, and validating the final state. This task demonstrates conditional logic, service management, error handling, and basic troubleshooting using Bash scripting on Amazon Linux 2023.

Task 7: Create a Bash Script for a Calculator

Outcome

A Bash script named calculator.bash is created to perform basic arithmetic operations: addition, subtraction, multiplication, and division. The script accepts two numbers and an operator from the user, calculates the result, and displays the output.

Calculator script created and executed successfully.

Basic arithmetic operations can be performed through user input.

Objective

To understand Bash scripting concepts such as variables, user input, conditional statements, arithmetic operations, and error handling by developing a simple command-line calculator.

This task demonstrates how Bash scripts can accept input, process data, and display results based on user-selected operations.

Prerequisites

- A running Amazon Linux 2023 EC2 instance.
- SSH access to the EC2 instance.
- Root privileges or a user account with appropriate permissions.
- Basic knowledge of Linux commands and Bash scripting.

Step-by-Step Implementation

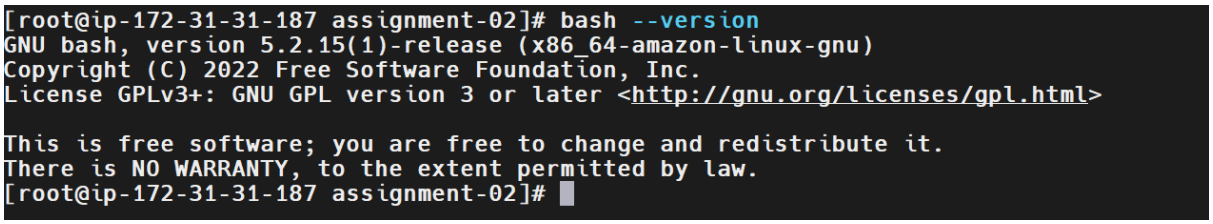
Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux EC2 instance through SSH.

Verify the Bash version:

bash --version

Screenshot Evidence



Step 2: Create the Calculator Script

Create a file named calculator.bash:

vim calculator.bash

Press i and enter the following script:

```
#!/bin/bash


echo "===== Bash Calculator ====="


read -p "Enter the first number: " num1

read -p "Enter an operator (+, -, *, /): " operator

read -p "Enter the second number: " num2


# Validate numeric inputs

if ! [[ "$num1" =~ ^-?[0-9]+([.][0-9]+)?$ ]] ||

  ! [[ "$num2" =~ ^-?[0-9]+([.][0-9]+)?$ ]]; then

   echo "Error: Please enter valid numbers."

   exit 1

fi


# Perform the calculation

case "$operator" in

   +)
```

```
    result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a+b}')
    ;;
  -)
    result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a-b}')
    ;;
  \*)
    result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a*b}')
    ;;
  /)
    if awk -v b="$num2" 'BEGIN {exit (b==0 ? 0 : 1)}'; then
      echo "Error: Division by zero is not allowed."
      exit 1
    fi
    result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a/b}')
    ;;
  *)
    echo "Error: Invalid operator."
    echo "Please use +, -, *, or /."
    exit 1
    ;;
esac

echo "Result: $num1 $operator $num2 = $result"
```

Save and exit Vim:

1. Press Esc.
2. Type :wq.
3. Press Enter.

Screenshot Evidence

```
#!/bin/bash

echo "===== Bash Calculator ====="

read -p "Enter the first number: " num1
read -p "Enter an operator (+, -, *, /): " operator
read -p "Enter the second number: " num2

# Validate numeric inputs
if ! [[ "$num1" =~ ^-?[0-9]+([.][0-9]+)?$ ]] ||
   ! [[ "$num2" =~ ^-?[0-9]+([.][0-9]+)?$ ]]; then
    echo "Error: Please enter valid numbers."
    exit 1
fi

# Perform the calculation
case "$operator" in
    +)
        result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a+b}')
        ;;
    -)
        result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a-b}')
        ;;
    \*)
        result=$(awk -v a="$num1" -v b="$num2" 'BEGIN {print a*b}')
        ;;
    /)
        if awk -v b="$num2" 'BEGIN {exit (b==0 ? 0 : 1)}'; then
            echo "Error: Division by zero is not allowed."
            exit 1
        fi
```

Step 3: Grant Execution Permission

Execute:

chmod 755 calculator.bash

This command grants the file owner read, write, and execute permissions and grants other users read and execute permissions.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# vim calculator.bash
[root@ip-172-31-31-187 assignment-02]# chmod 755 calculator.bash
```

Step 4: Execute the Calculator

Run the script:

./calculator.bash

Enter the requested values when prompted.

Example 1: Addition

Input:

Enter the first number: 10

Enter an operator (+, -, *, /): +

Enter the second number: 9

Expected output:

Result: 10 + 9 = 19

Example 2: Subtraction

Input:

Enter the first number: 5

Enter an operator (+, -, *, /): -

Enter the second number: 4

Expected output:

Result: 5-4 = 1

Example 3: Multiplication

Input:

Enter the first number: 10

Enter an operator (+, -, *, /): *

Enter the second number: 5

Expected output:

Result: 10 * 5 = 50

Example 4: Division

Input:

Enter the first number: 10

Enter an operator (+, -, *, /): /

Enter the second number: 5

Expected output:

Result: 10 / 5 = 2

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 10
Enter an operator (+, -, *, /): +
Enter the second number: 9
Result: 10 + 9 = 19
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 5
Enter an operator (+, -, *, /): -
Enter the second number: 4
Result: 5 - 4 = 1
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 10
Enter an operator (+, -, *, /): *
Enter the second number: 5
Result: 10 * 5 = 50
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 10
Enter an operator (+, -, *, /): /
Enter the second number: 5
Result: 10 / 5 = 2
```

Step 5: Test Error Handling

Test division by zero:

Enter the first number: 10

Enter an operator (+, -, *, /): /

Enter the second number: 0

Expected output:

Error: Division by zero is not allowed.

Test an invalid operator:

Enter the first number: 10

Enter an operator (+, -, *, /): %

Enter the second number: 5

Expected output:

Error: Invalid operator.

Please use +, -, *, or /.

Test invalid numeric input:

Enter the first number: abc

Expected output:

Error: Please enter valid numbers.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 10
Enter an operator (+, -, *, /): /
Enter the second number: 0
Error: Division by zero is not allowed.
[root@ip-172-31-31-187 assignment-02]# ./calculator.bash
===== Bash Calculator =====
Enter the first number: 10
Enter an operator (+, -, *, /): &
Enter the second number: 9
Error: Invalid operator.
Please use +, -, *, or /.
[root@ip-172-31-31-187 assignment-02]#
```

Validation Steps

Verify that the calculator performs all four arithmetic operations correctly.

Test case	Input	Expected result
Addition	10 + 5	15
Subtraction	10 - 5	5
Multiplication	10 * 5	50
Division	10 / 5	2
Division by zero	10 / 0	Error message
Invalid operator	10 % 5	Error message

Explanation

The calculator uses the read command to collect two numbers and an operator from the user. The case statement selects the appropriate arithmetic operation based on the operator entered. The awk command performs the calculations, allowing the script to support decimal as well as integer values.

The script also validates the input numbers, checks for division by zero, and rejects unsupported operators. These checks help prevent incorrect calculations and demonstrate the importance of input validation and error handling in Bash scripting.

Real-Time Use Case

Bash calculators can be useful for simple administrative calculations within shell scripts, such as calculating storage differences, converting numeric values, or performing arithmetic during automation tasks.

Although this example is intended for learning, the same concepts—user input, conditional logic, validation, and calculation—are used in more complex DevOps automation scripts.

Common Errors and Troubleshooting

Error	Cause	Resolution

Permission denied	The script does not have execution permission.	Run chmod 755 calculator.bash.
awk: command not found	The required utility is unavailable.	Install the appropriate gawk package if needed using dnf install -y gawk.
Invalid operator error	An unsupported operator was entered.	Use +, -, *, or /.
Division by zero error	The second number is zero during division.	Enter a non-zero second number.
Invalid number error	The input contains non-numeric characters.	Enter valid integer or decimal values.
Script not found	The command was executed from a different directory.	Run pwd and ls -l calculator.bash to locate the file.

Conclusion

The calculator.bash script demonstrates how Bash can accept user input, perform arithmetic operations, and handle common errors. The task provides practical experience with variables, read, case, regular-expression input validation, and the awk command.

The calculator can be extended in the future to support additional operations, repeated calculations, and a menu-driven interface.

Task 8: Check If a Directory Exists and Create It If Not

Outcome

A Bash script named check_create_directory.bash is created to check whether a specified directory exists. If the directory exists, the script displays a confirmation message. If the directory does not exist, the script creates it and verifies successful creation.

Directory existence checked successfully.

If the directory was missing, it was created automatically.

Objective

To automate directory management using Bash scripting. This task helps develop practical knowledge of conditional statements, directory checks, and filesystem operations in Linux.

Checking for a directory before creating it prevents unnecessary creation attempts and helps scripts manage required directories reliably.

Prerequisites

- An AWS account with a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.

- Root privileges or appropriate directory permissions.
- Basic knowledge of Linux commands and Bash scripting.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux EC2 instance using SSH.

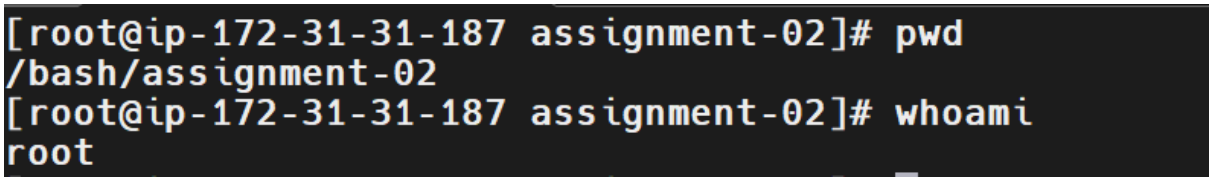
Verify the current working directory:

pwd

Check the current user:

whoami

Screenshot Evidence



Step 2: Create the Bash Script

Create a file named check_create_directory.bash:

vim check_create_directory.bash

Press i and enter the following script:

```
#!/bin/bash

DIR="/tmp/devops-directory"

echo "Checking directory: $DIR"

if [ -d "$DIR" ]; then
   echo "Directory already exists: $DIR"
else
   echo "Directory does not exist. Creating it..."

   if mkdir -p "$DIR"; then
      echo "Directory created successfully: $DIR"
   else
```

```
        echo "Error: Failed to create directory."
        exit 1
    fi
fi


if [ -d "$DIR" ]; then
    echo "Validation successful: Directory exists."
else
    echo "Validation failed: Directory does not exist."
    exit 1
fi
```

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Step 3: Grant Execution Permission

Execute:

chmod 755 check_create_directory.bash

This command grants execution permission to the script.

Screenshot Evidence



Step 4: Execute the Script When the Directory Does Not Exist

First, remove the practice directory if it already exists:

rm -rf /tmp/devops-directory

Caution: This command permanently removes the specified directory and its contents. Use it only for this disposable practice directory.

Run the script:

./check_create_directory.bash

Expected output:

Checking directory: /tmp/devops-directory

Directory does not exist. Creating it...

Directory created successfully: /tmp/devops-directory

Validation successful: Directory exists.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# ./check_directory.bash
Checking directory: /tmp/devops-directory
Directory does not exist. Creating it...
Directory created successfully: /tmp/devops-directory
Validation successful: Directory exists.
[root@ip-172-31-31-187 assignment-02]#
```

Step 5: Execute the Script When the Directory Already Exists

Run the script again:

./check_create_directory.bash

Expected output:

Checking directory: /tmp/devops-directory

Directory already exists: /tmp/devops-directory

Validation successful: Directory exists.

This confirms that the script detects an existing directory without attempting to create it again.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# ./check_directory.bash
Checking directory: /tmp/devops-directory
Directory already exists: /tmp/devops-directory
Validation successful: Directory exists.
[root@ip-172-31-31-187 assignment-02]#
```

Validation Steps

Step 1: Verify the Directory

Execute:

ls -ld /tmp/devops-directory

The output should display the directory's permissions, ownership, and name.

Step 2: Check the Directory Using a Conditional Statement

Execute:

```
if [ -d /tmp/devops-directory ]; then
   echo "Directory exists"
else
   echo "Directory does not exist"
fi
```

Expected output:

Directory exists

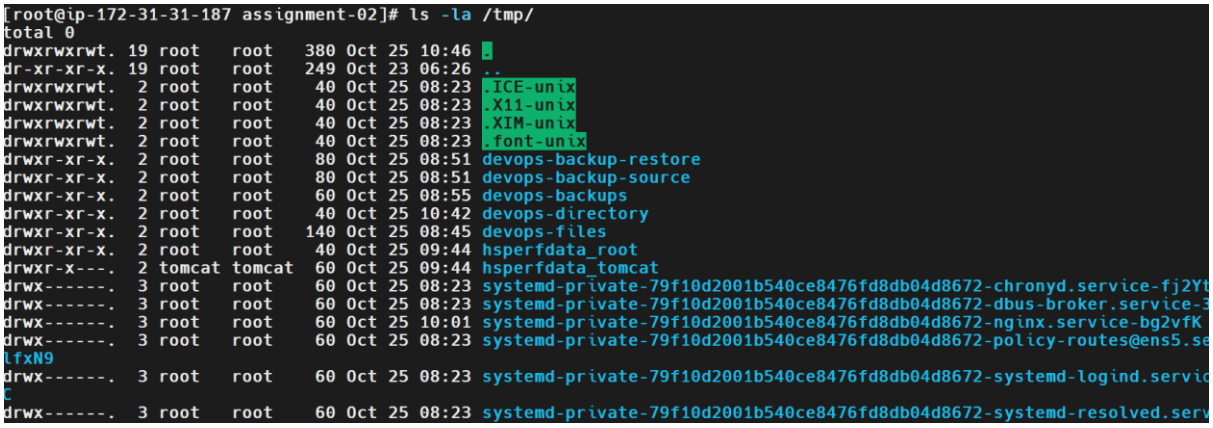
Step 3: Verify the Script File

Execute:

ls -l check_create_directory.bash

Confirm that the script has execute permissions.

Validation Screenshot



Explanation

The script uses the -d conditional test to determine whether the specified path exists and is a directory. If the condition is true, the script displays a message confirming that the directory already exists. Otherwise, it executes mkdir -p to create the directory.

The script then checks the directory again to confirm that the operation succeeded. The if statement provides decision-making logic, while mkdir -p safely creates the specified directory and any missing parent directories without reporting an error merely because the directory already exists.

Real-Time Use Case

DevOps engineers frequently need to ensure that directories exist before running automation tasks. For example, a backup script may need a destination directory, a deployment script may require an application directory, and a logging script may need a directory for storing log files.

By checking and creating directories automatically, Bash scripts reduce manual intervention and help make automation more reliable and repeatable.

Common Errors and Troubleshooting

Error	Cause	Resolution
Permission denied	The user lacks permission to create a directory at the specified path.	Use a directory you own or an authorized account with appropriate permissions.
No such file or directory	A parent directory is missing or the specified path is incorrect.	Verify the path and use mkdir -p to create missing parent directories.
command not found	A command was typed incorrectly or is unavailable.	Check the spelling and command availability.

Script does not execute	Execution permission is missing.	Run chmod 755 check_create_directory.bash.
Directory already exists	The target directory was created during an earlier run.	This is expected; the script should display the existing-directory message.
Directory creation fails	Filesystem permissions or storage limitations prevent creation.	Check directory permissions, disk space, and the exact error message.

Conclusion

The check_create_directory.bash script demonstrates how to check directory existence, create a missing directory, and validate the result using Bash scripting on Amazon Linux 2023.

This task provides practical experience with conditional statements, filesystem checks, directory creation, and error handling. These techniques are useful in backup automation, application deployments, and other DevOps workflows.

Task 9: Create a Bash Script to Delete the Last 3 Lines of a File

Outcome

A Bash script named delete_last3_lines.bash is created to remove the last three lines from a specified file. The script validates the input file, creates a backup before modifying it, removes the last three lines, and displays the updated file content.

The last three lines of the specified file are removed successfully.

A backup of the original file is retained for recovery.

Objective

To learn how to manipulate text files using Bash scripting and Linux command-line utilities. This task demonstrates file validation, backup creation, temporary files, and the use of sed to remove specific lines from a file.

This technique is useful when automating log cleanup, editing configuration files, and processing text files.

Prerequisites

- An AWS account with a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.
- Root privileges or appropriate file permissions.
- Basic knowledge of Bash scripting and Linux commands.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux EC2 instance using SSH.

Verify the working directory:

pwd

Check the available files:

ls -l

```
[root@ip-172-31-31-187 assignment-02]# pwd
/bash/assignment-02
[root@ip-172-31-31-187 assignment-02]# whoami
root
```

Step 2: Create a Sample File

Create a sample file named sample.txt:

cat > sample.txt <<'EOF'

Line 1: Introduction

Line 2: Linux

Line 3: Bash scripting

Line 4: File handling

Line 5: Backup

Line 6: Automation

Line 7: DevOps

Line 8: Conclusion

EOF

Display the original file:

cat -n sample.txt

Expected output:

1 Line 1: Introduction
2 Line 2: Linux
3 Line 3: Bash scripting
4 Line 4: File handling
5 Line 5: Backup
6 Line 6: Automation
7 Line 7: DevOps
8 Line 8: Conclusion

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# cat sample.txt
Line 1: Introduction
Line 2: Linux
Line 3: Bash scripting
Line 4: File handling
Line 5: Backup
Line 6: Automation
Line 7: DevOps
Line 8: Conclusion
```

Step 3: Create the Bash Script

Create a script named delete_last3_lines.bash:

vim delete_last3_lines.bash

Enter the following code:

```
#!/bin/bash

FILE="${1:-sample.txt}"

echo "Checking file: $FILE"

if [ ! -f "$FILE" ]; then
    echo "Error: File does not exist."
    exit 1
fi

if [ ! -r "$FILE" ] || [ ! -w "$FILE" ]; then
    echo "Error: File must be readable and writable."
    exit 1
fi

BACKUP="${FILE}.bak"
TEMP_FILE=$(mktemp) || {
    echo "Error: Unable to create temporary file."
    exit 1
}
```

```
trap 'rm -f "$TEMP_FILE"' EXIT

if ! cp -p -- "$FILE" "$BACKUP"; then
    echo "Error: Backup creation failed."
    exit 1
fi

if ! sed '$d' "$FILE" | sed '$d' | sed '$d' > "$TEMP_FILE"; then
    echo "Error: Failed to process the file."
    exit 1
fi

if ! cat "$TEMP_FILE" > "$FILE"; then
    echo "Error: Failed to update the file."
    exit 1
fi

echo "Last three lines deleted successfully."
echo "Backup created: $BACKUP"
echo "Updated file content:"
cat -n "$FILE"
```

Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

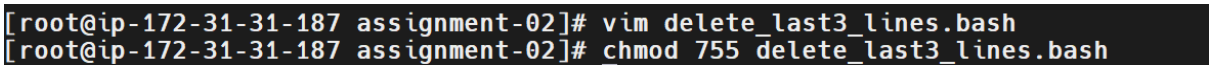
Step 4: Grant Execution Permission

Run:

chmod 755 delete_last3_lines.bash

This grants the file owner read, write, and execute permissions, while other users receive read and execute permissions.

Screenshot Evidence



Step 5: Execute the Script

Run:

./delete_last3_lines.bash sample.txt

The script creates a backup named sample.txt.bak and removes the last three lines from sample.txt.

Expected output:

Checking file: sample.txt

Last three lines deleted successfully.

Backup created: sample.txt.bak

Updated file content:

1 Line 1: Introduction

2 Line 2: Linux

3 Line 3: Bash scripting

4 Line 4: File handling

5 Line 5: Backup

```
[root@ip-172-31-31-187 assignment-02]# cat -n sample.txt.bak
     1  Line 1: Introduction
     2  Line 2: Linux
     3  Line 3: Bash scripting
     4  Line 4: File handling
     5  Line 5: Backup
```

Step 6: Verify the Backup

Run:

cat -n sample.txt.bak

The backup should contain all eight original lines.

Verify the updated file:

cat -n sample.txt

The updated file should contain only the first five lines.

Screenshot Evidence

```
[root@ip-172-31-31-187 assignment-02]# cat -n sample.txt.bak
     1  Line 1: Introduction
     2  Line 2: Linux
     3  Line 3: Bash scripting
     4  Line 4: File handling
     5  Line 5: Backup
```

Validation Steps

Step 1: Count the Remaining Lines

Run:

wc -l sample.txt

Expected output:

5 sample.txt

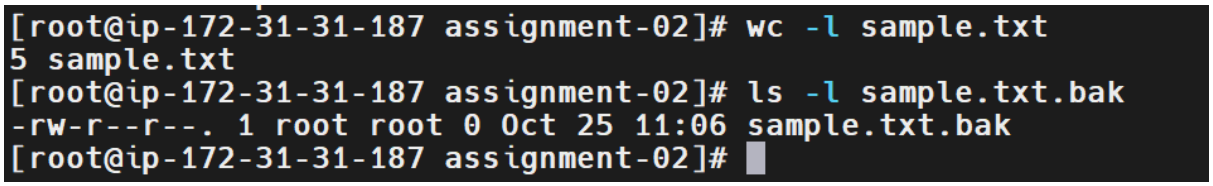
Step 2: Confirm the Backup Exists

Run:

ls -l sample.txt.bak

Verify that the backup file exists.

Validation Screenshot



Explanation

The script checks whether the specified file exists and is readable and writable before modifying it. It creates a backup using cp -p, preserving the original file's contents and basic file attributes. It then uses three successive sed '$d' operations to remove the final line three times. The resulting content is written to a temporary file before replacing the original file's contents.

The script accepts a filename as a command-line argument. For example, ./delete_last3_lines.bash sample.txt processes sample.txt. If no filename is supplied, it defaults to sample.txt. This makes the script reusable for different files while retaining a backup of the original content.

Real-Time Use Case

DevOps engineers regularly automate text-file processing, configuration management, and log maintenance. Removing unwanted trailing lines can be useful when cleaning generated files or preparing text data for further processing.

In production environments, files should be backed up before modification, and critical configuration files should be validated after changes. For large or important files, a safer production implementation would also use atomic replacement and preserve ownership, permissions, and metadata.

Common Errors and Troubleshooting

Error	Cause	Resolution
File does not exist	Incorrect filename or file path.	Check the path using ls -l and pass the correct filename.
Permission denied	The current user lacks the required file permissions.	Check ownership and permissions; use an authorized account.

Script does not execute	Execute permission is missing.	Run chmod 755 delete_last3_lines.bash.
Backup creation failed	Insufficient permissions or disk space.	Check directory permissions and available space.
File becomes empty	The file originally contained three or fewer lines.	Verify the backup before restoring if needed.
Unexpected file contents	Incorrect filename or repeated execution.	Inspect the backup and use a fresh sample file for testing.

Conclusion

The delete_last3_lines.bash script demonstrates file validation, backup creation, text processing with sed, and temporary-file handling. It removes the last three lines from a specified file and retains a backup of the original content.

This task provides practical experience with Bash scripting and Linux file manipulation, which are useful skills for DevOps automation and routine system administration.

Task 10: Monitor CPU Usage and Send an Email Notification if It Exceeds 80%

Outcome

A Bash script named monitor_cpu.bash is created to monitor CPU utilization on an Amazon Linux 2023 EC2 instance. If CPU usage exceeds 80%, the script attempts to send an email notification to the configured recipient.

CPU utilization is monitored using a Bash script.

When CPU usage exceeds 80%, an email alert is triggered.

Note: Actual email delivery requires a configured email-sending method and valid SMTP or mail relay settings. Executing the script alone does not guarantee that an email will be delivered.

Objective

To automate CPU monitoring and alerting using Bash scripting. High CPU utilization can affect application response times, server performance, and service availability. An automated alert helps administrators identify potential performance issues and investigate them before they affect users.

This task demonstrates CPU metric collection, conditional statements, threshold comparison, and email notification integration on Linux.

Prerequisites

- An AWS account with a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.
- Root privileges or appropriate permissions.
- Basic knowledge of Bash scripting.

- The top or mpstat utility for collecting CPU utilization.
- A working email-sending method, such as a configured SMTP relay or an approved email service.
- A valid recipient email address.
- Network access to the configured email service.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux 2023 EC2 instance through SSH.

Verify the operating system:

cat /etc/os-release

Check the CPU information:

lscpu

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 46 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          2
  On-line CPU(s) list:           0,1
Vendor ID:                       GenuineIntel
  BIOS Vendor ID:                Intel(R) Corporation
  Model name:                    Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
    BIOS Model name:             Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
    CPU family:                  6
    Model:                       85
    Thread(s) per core:          2
    Core(s) per socket:          1
    Socket(s):                   1
    Stepping:                    7
    BogoMIPS:                    4999.99
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush
                                 se2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology nons
                                 tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popc
                                 e_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch cpuid_fau
                                 e tsc_adjust bmi1 avx2 smep bmi2 erms invpcid mpx avx512f avx512dq rdseed adx sm
                                 clwb avx512cd avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves ida arat pku ospk
Virtualization features:
  Hypervisor vendor:             KVM
  Virtualization type:           full
Caches (sum of all):
  L1d:                           32 KiB (1 instance)
```

Step 2: Install the Required Utilities

Update the package metadata:

dnf makecache

Install the sysstat package, which provides the mpstat command:

dnf install -y sysstat

Verify the installation:

mpstat -V

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# dnf makecache
Amazon Linux 2023 repository                                    54 kB/s | 3.6 kB     00:00
Amazon Linux 2023 Kernel Livepatch repository                   46 kB/s | 2.9 kB     00:00
Metadata cache created.
[root@ip-172-31-31-187 ec2-user]# dnf install -y sysstat
Last metadata expiration check: 0:00:10 ago on Sun Oct 25 11:24:58 2026.
Package sysstat-12.5.6-1.amzn2023.0.3.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-31-187 ec2-user]# mpstat -V
sysstat version 12.5.6
(C) Sebastien Godard (sysstat <at>_orange.fr)
```

Step 3: Check CPU Utilization Manually

Run:

mpstat 1 1

Look for the %idle column in the output.

CPU utilization can be calculated as:

CPU Usage (%) = 100 - %idle

For example, if %idle is 15.00, CPU usage is:

100 - 15 = 85%

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# mpstat 1 1
Linux 6.18.54-121.166.amzn2023.x86_64 (ip-172-31-31-187.ec2.internal)   10/25/26        _x86_64_        (2 CPU)

11:26:10     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
11:26:11     all    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
Average:     all    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
```

Step 4: Configure an Email Notification Method

Before running the monitoring script, configure an approved email-sending method.

For example, an organization may use an SMTP relay or Amazon Simple Email Service (Amazon SES). The sending method must be configured with valid credentials, an authorized sender address, and an approved recipient where required.

Do not put SMTP passwords or access keys directly in the Bash script.

For this task, the script will use a local mail command supplied by a configured mail-transfer service or relay.

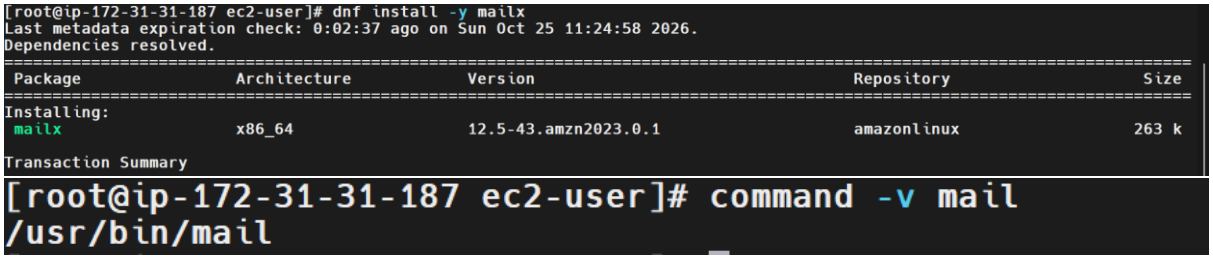
Check whether the command is available:

command -v mail

If the command is missing, install an appropriate mail client and configure its sending service according to your organization's environment. Installing a mail client alone does not configure email delivery.

Test the configured email method before proceeding.
Installed mail client by using command dnf install -y mailx

Screenshot Evidence



Step 5: Create the Bash Script

Create a file named monitor_cpu.bash:

vim monitor_cpu.bash

Press i and enter the following script:

```
#!/bin/bash

THRESHOLD=80
EMAIL="admin@example.com"

# Collect CPU utilization
IDLE=$(mpstat 1 1 | awk '/Average:/ && $NF ~ /^[0-9.]+$/ {print $NF}')

if ! [[ "$IDLE" =~ ^[0-9]+([.][0-9]+)?$ ]]; then
   echo "Error: Unable to determine CPU idle percentage."
   exit 1
fi

# Calculate CPU usage
CPU_USAGE=$(awk -v idle="$IDLE" 'BEGIN {
   printf "%.2f", 100 - idle
}')

echo "Current CPU Usage: ${CPU_USAGE}%"
echo "CPU Threshold: ${THRESHOLD}%"

# Compare CPU usage with the threshold
```

```
if awk -v usage="$CPU_USAGE" -v threshold="$THRESHOLD" \
   'BEGIN {exit !(usage > threshold)}'; then

   SUBJECT="High CPU Usage Alert"
   BODY="Warning: CPU usage on $(hostname) is ${CPU_USAGE}%, exceeding the threshold of ${THRESHOLD}%."

   echo "$BODY" | mail -s "$SUBJECT" "$EMAIL"

   if [ $? -eq 0 ]; then
      echo "CPU usage exceeded the threshold. Email command completed."
   else
      echo "Error: Email notification command failed."
      exit 1
   fi
else
   echo "CPU usage is within the configured threshold."
fi
```

Replace admin@example.com with your actual recipient email address.

Save the file by pressing Esc, typing :wq, and pressing Enter.

Screenshot Evidence

Insert a screenshot showing the completed script in Vim.

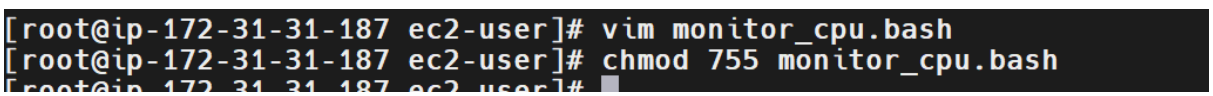
Step 6: Grant Execution Permission

Run:

chmod 755 monitor_cpu.bash

This grants execute permission to the script.

Screenshot Evidence



Step 7: Execute the Script

Run:

./monitor_cpu.bash

If CPU usage is below or equal to 80%, the output should resemble:

Current CPU Usage: 25.00%

CPU Threshold: 80%

CPU usage is within the configured threshold.

If CPU usage exceeds 80%, the script attempts to send an email notification.

Example output:

Current CPU Usage: 85.00%

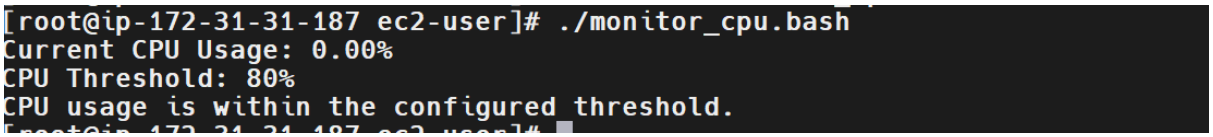
CPU Threshold: 80%

CPU usage exceeded the threshold. Email command completed.

The example percentages are illustrative; actual CPU utilization depends on the workload at execution time.

Important: The message Email command completed means the local mail command returned success. It does not independently confirm that the recipient received the email.

Screenshot Evidence



Step 8: Test the High-CPU Alert

CPU usage on a lightly loaded EC2 instance may never exceed 80%. If you need to test the alert, use a controlled test environment and a temporary CPU workload.

For example, install the stress-ng utility if it is available from your approved repositories:

dnf install -y stress-ng

Check available CPU cores:

nproc

Generate a temporary CPU workload for a limited period:

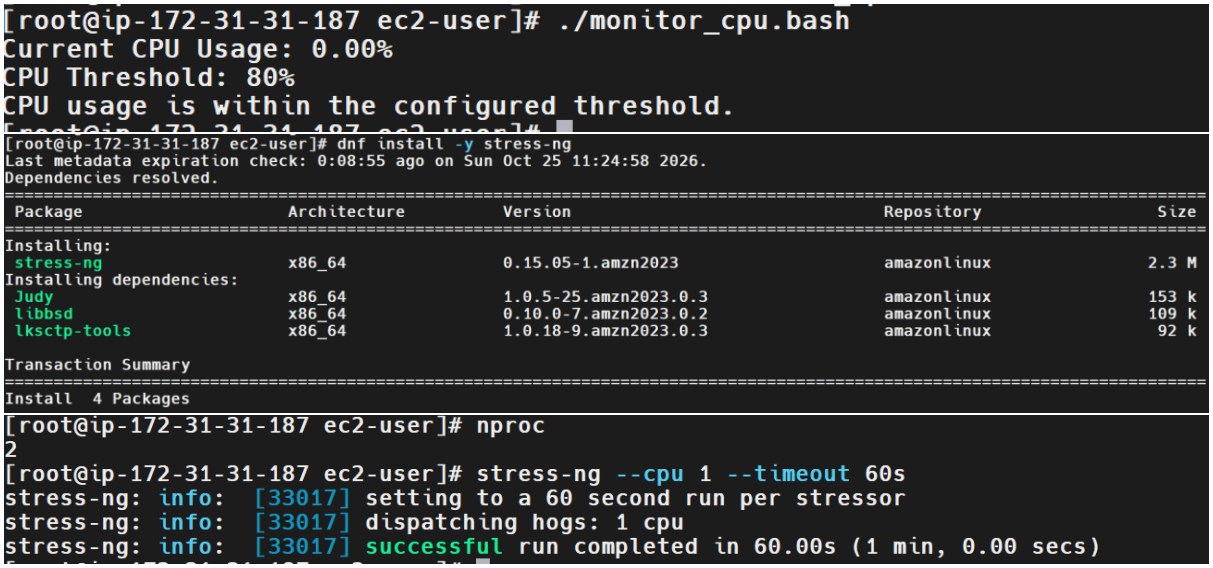
stress-ng --cpu 1 --timeout 60s

While the workload is running, execute the monitoring script in another terminal:

./monitor_cpu.bash

The workload may not push total CPU usage above 80%, especially on an instance with multiple vCPUs. Adjust the workload carefully in the lab and stop it when testing is complete.

Screenshot Evidence



Validation Steps

Step 1: Verify CPU Metrics

Run:

mpstat 1 1

Confirm that the script is able to read the CPU idle percentage and calculate CPU usage.

Step 2: Verify the Threshold Logic

Confirm that the script sends an alert only when CPU usage is strictly greater than 80%.

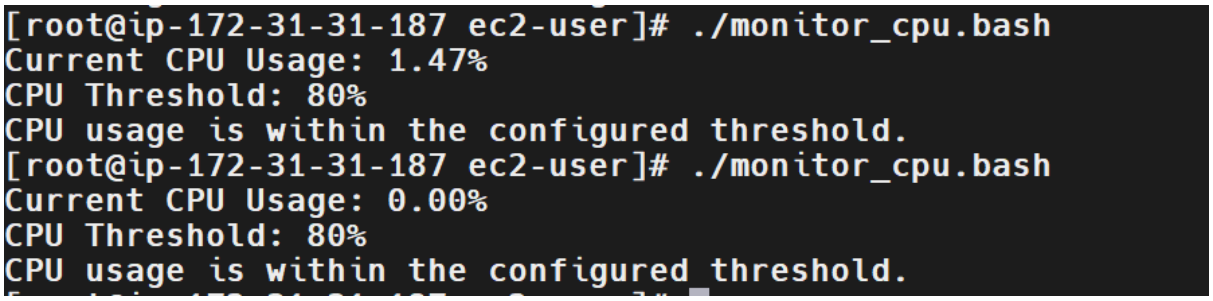
CPU usage	Expected behavior
45%	No alert
80%	No alert
85%	Email notification attempted

Step 3: Verify Email Delivery

Check the configured mail service's logs or delivery status, and confirm that the recipient received the test email.

A successful script execution alone is not sufficient to verify email delivery.

Validation Screenshot



Explanation

The script uses mpstat to collect CPU statistics. The %idle value indicates the percentage of time the CPU was idle, so subtracting it from 100 provides an estimate of CPU utilization. The script then compares the calculated utilization with the configured threshold of 80%.

When CPU utilization is greater than 80%, the script constructs an alert message containing the server hostname and current CPU usage. It then invokes the configured mail command to send the message. If utilization does not exceed the threshold, no email is attempted. This provides a basic monitoring and notification workflow using Bash.

Real-Time Use Case

DevOps and system administration teams monitor CPU utilization on application servers, database servers, and cloud instances to detect performance bottlenecks. High CPU utilization can result from heavy traffic, inefficient application code, runaway processes, or background jobs.

A monitoring script can provide an additional alerting mechanism for small environments or learning labs. In production environments, continuous monitoring should generally use a dedicated monitoring platform, such as Amazon CloudWatch, Prometheus, or another observability solution, with configured notification channels.

Common Errors and Troubleshooting

Error	Cause	Resolution
mpstat: command not found	The sysstat package is missing.	Install it using dnf install -y sysstat.
CPU utilization cannot be calculated	The mpstat output differs from what the script expects.	Run mpstat 1 1 manually and inspect its output.
mail: command not found	A mail client is not installed.	Install an appropriate mail client and configure a mail relay.
Email is not received	SMTP settings, sender authorization, or network connectivity is incorrect.	Check the email service configuration and delivery logs.
CPU usage never exceeds 80%	The EC2 instance has a light workload.	Test with a controlled, temporary CPU workload.
Email command succeeds but no message arrives	Local acceptance does not guarantee final delivery.	Verify relay logs and recipient delivery status.
Script permission denied	The script is not executable or the user lacks permission.	Run chmod 755 monitor_cpu.bash and use an authorized account.

Conclusion

The monitor_cpu.bash script demonstrates how to collect CPU utilization, calculate usage, compare it against an 80% threshold, and attempt an email notification when the threshold is exceeded.

This task provides practical experience with Bash scripting, system monitoring, conditional logic, and alerting. For reliable production monitoring, the script should be combined with a properly configured email service, delivery verification, and a dedicated monitoring and alerting platform.

Task 11: Monitor Disk Space and Send an Email Notification if Usage Exceeds 80%

Outcome

A Bash script named monitor_disk.bash is created to monitor disk space utilization on an Amazon Linux 2023 EC2 instance. If the usage of a monitored filesystem exceeds 80%, the script attempts to send an email notification to the configured recipient.

Disk space utilization is monitored using a Bash script.

If a monitored filesystem exceeds 80% usage, an email alert is triggered.

Note: Email delivery requires a configured mail-transfer service or SMTP relay. The mail command being installed does not guarantee that an email will be delivered.

Objective

To automate disk space monitoring and email alerting using Bash scripting. When a filesystem becomes nearly full, applications may fail to write files, logs may stop recording events, and services may become unstable. Monitoring disk utilization helps administrators identify storage problems before they affect system availability.

Prerequisites

- An AWS account with a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.
- Root privileges or appropriate permissions.
- Basic knowledge of Bash scripting and Linux commands.
- The df utility for checking filesystem usage.
- The mail command installed.
- A configured SMTP relay or mail-transfer service.
- A valid recipient email address.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux EC2 instance using SSH.

Verify the operating system:

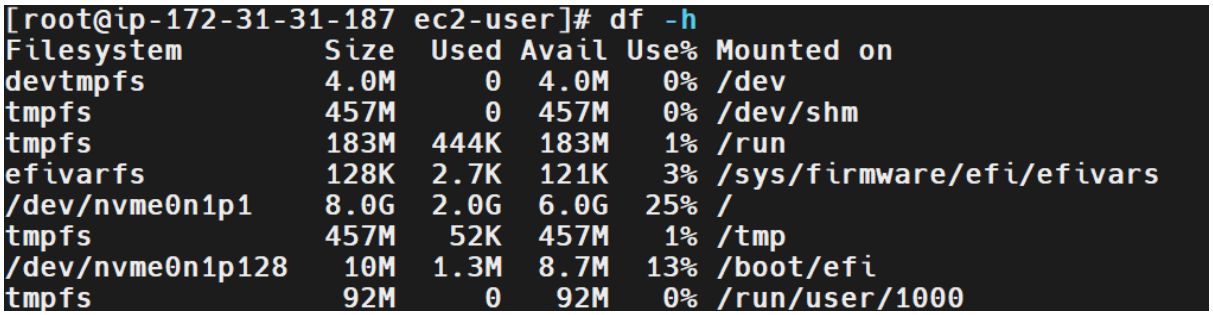
cat /etc/os-release

Check available disk space:

df -h

The df -h command displays filesystem capacity, used space, available space, and utilization percentage in a human-readable format.

Screenshot Evidence



Step 2: Verify the Email Command

Check whether the mail command is available:

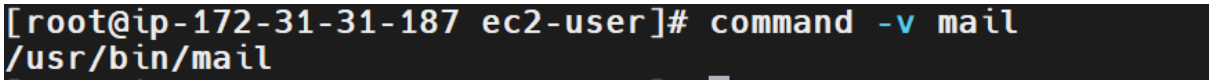
command -v mail

Expected output:

/usr/bin/mail

This confirms that the command is available. Actual email delivery still requires a properly configured mail service or SMTP relay.

Screenshot Evidence



Step 3: Create the Bash Script

Create a file named monitor_disk.bash:

vim monitor_disk.bash

Press i and enter the following script:

#!/bin/bash

THRESHOLD=80

EMAIL="admin@example.com"

echo "Checking disk space utilization..."

df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {print $1, $6, $5}' |

while read -r FILESYSTEM MOUNTPOINT USAGE; do

```
    USAGE=${USAGE%\%}

    # Ignore entries without a numeric usage value
    if ! [[ "$USAGE" =~ ^[0-9]+$ ]]; then
        continue
    fi

    echo "Filesystem: $FILESYSTEM"
    echo "Mount Point: $MOUNTPOINT"
    echo "Usage: ${USAGE}%"

    if [ "$USAGE" -gt "$THRESHOLD" ]; then

        SUBJECT="High Disk Usage Alert: $MOUNTPOINT"

        BODY="Warning: Disk usage on $(hostname) has exceeded ${THRESHOLD}%.

Filesystem: $FILESYSTEM
Mount Point: $MOUNTPOINT
Current Usage: ${USAGE}%

Please investigate disk usage."

        if echo "$BODY" | mail -s "$SUBJECT" "$EMAIL"; then
            echo "Email command completed for $MOUNTPOINT."
        else
            echo "Error: Failed to submit email for $MOUNTPOINT."
        fi
    else
        echo "Disk usage is within the configured threshold."
```

fi

echo "----------"

done

Replace admin@example.com with your actual recipient email address.

Save and exit Vim:

1. Press Esc.
2. Type :wq.
3. Press Enter.

Screenshot Evidence

```
#!/bin/bash

THRESHOLD=80
EMAIL="naveenlingampelli456@gmail.com"

echo "Checking disk space utilization..."

df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {print $1, $6, $5}' |
while read -r FILESYSTEM MOUNTPOINT USAGE; do

    USAGE=${USAGE%\%}

    # Ignore entries without a numeric usage value
    if ! [[ "$USAGE" =~ ^[0-9]+$ ]]; then
        continue
    fi

    echo "Filesystem: $FILESYSTEM"
    echo "Mount Point: $MOUNTPOINT"
    echo "Usage: ${USAGE}%"

    if [ "$USAGE" -gt "$THRESHOLD" ]; then

        SUBJECT="High Disk Usage Alert: $MOUNTPOINT"

        BODY="Warning: Disk usage on $(hostname) has exceeded ${THRESHOLD}%.

Filesystem: $FILESYSTEM
Mount Point: $MOUNTPOINT
Current Usage: ${USAGE}%
```

Step 4: Grant Execution Permission

Run:

chmod 755 monitor_disk.bash

This grants the owner read, write, and execute permissions, and other users read and execute permissions.

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# vim monitor_disk.bash
[root@ip-172-31-31-187 ec2-user]# chmod 755 monitor_disk.bash
```

Step 5: Execute the Script

Run:

./monitor_disk.bash

Example output when disk utilization is below the threshold:

Checking disk space utilization...

Filesystem: /dev/nvme0n1p1

Mount Point: /

Usage: 45%

Disk usage is within the configured threshold.

The filesystem name and utilization percentage above are illustrative. Your actual output will depend on your EC2 instance.

If a filesystem exceeds 80%, the script attempts to send an email notification containing the filesystem name, mount point, and current utilization.

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# ./monitor_disk.bash
Checking disk space utilization...
Filesystem: efivarfs
Mount Point: /sys/firmware/efi/efivars
Usage: 3%
Disk usage is within the configured threshold.
-----------------------------
Filesystem: /dev/nvme0n1p1
Mount Point: /
Usage: 25%
Disk usage is within the configured threshold.
-----------------------------
Filesystem: /dev/nvme0n1p128
Mount Point: /boot/efi
Usage: 13%
Disk usage is within the configured threshold.
```

Step 6: Test the Email Notification

First, ensure that your email-sending service is configured and working.

You can test the mail command with:

echo "Disk monitoring test email" | mail -s "Disk Alert Test" your-email@example.com

Replace your-email@example.com with the recipient address.

To test the alert condition without filling the disk, you can temporarily change the threshold in the script to a lower value, such as THRESHOLD=1, and run it again. This should trigger alerts for filesystems whose usage exceeds 1%.

After testing, restore the threshold to:

THRESHOLD=80

Important: The script's message that an email command completed does not confirm that the recipient received the email. Verify delivery through the inbox or the configured mail service's logs.

Screenshot Evidence

Validation Steps

Step 1: Verify Disk Usage

Run:

df -h

Confirm the utilization percentage for each filesystem.

Step 2: Verify the Script

Run:

bash -n monitor_disk.bash

If the command produces no output, the script has passed Bash syntax checking.

Step 3: Verify the Alert Condition

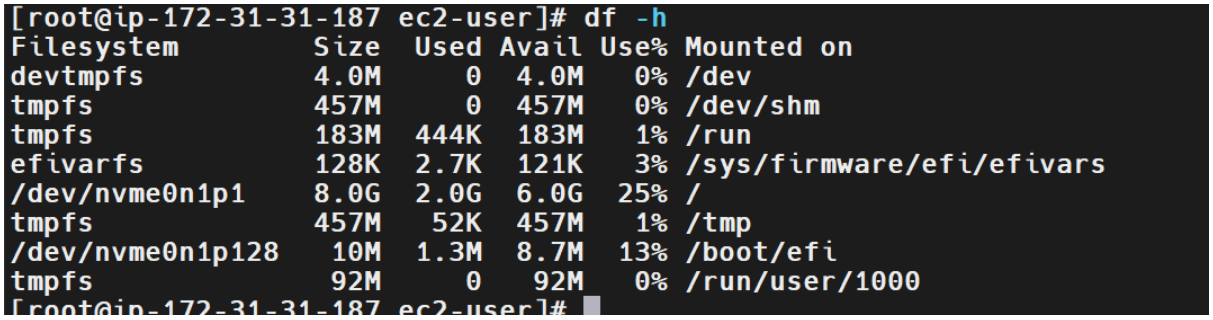
Confirm that the script attempts to send an email only when filesystem usage is strictly greater than 80%.

Disk usage	Expected behavior
40%	No email alert
80%	No email alert
85%	Email notification attempted

Step 4: Verify Email Delivery

Check the recipient's inbox or the configured mail service's delivery logs. A successful local command alone is not sufficient proof of delivery.

Validation Screenshot



Explanation

The script uses the df command to collect filesystem usage information. The -P option requests a predictable output format, while -x tmpfs -x devtmpfs excludes temporary memory-backed filesystems from monitoring. The awk command extracts the filesystem name, mount point, and percentage used.

The script removes the percent sign from the usage value and compares the resulting integer with the configured threshold of 80%. When utilization is greater than 80%, the script constructs an alert message and submits it using the mail command. Otherwise, it reports that disk usage is within the threshold.

Real-Time Use Case

DevOps engineers monitor disk utilization on application servers, database servers, and cloud instances to prevent storage exhaustion. High disk usage may result from growing application logs, backup files, temporary files, or large application data.

An automated disk monitoring script can help identify potential storage issues before applications fail. In production environments, this monitoring can be integrated with Amazon CloudWatch, Prometheus, or another monitoring platform to provide centralized dashboards and reliable alerting.

Common Errors and Troubleshooting

Error	Cause	Resolution
df: command not found	The required utility is unavailable or the environment is misconfigured.	Verify the command path and operating system utilities.
mail: command not found	The mail client is not installed.	Install an appropriate mail client and configure a delivery service.
Email is not received	SMTP settings, sender authorization, or network connectivity is incorrect.	Check relay configuration and delivery logs.
No alert is triggered	Disk usage has not exceeded 80%.	Check actual utilization using df -h; use a lower temporary threshold for testing.
Incorrect filesystem output	Filesystem types or mount points differ from expectations.	Review the output of df -P and adjust exclusions if necessary.

Script permission denied	Execution permission is missing.	Run chmod 755 monitor_disk.bash.
Disk is nearly full	Logs, backups, or application data consume available space.	Identify large files and directories, then clean up safely or expand storage.

Conclusion

The monitor_disk.bash script demonstrates how to monitor filesystem utilization, compare usage against an 80% threshold, and attempt email notification when the threshold is exceeded.

This task provides practical experience with Bash scripting, filesystem monitoring, conditional logic, and email alert integration. Reliable production alerting requires a configured email delivery service and should ideally be combined with a dedicated monitoring platform.

Task 12: Monitor Memory Usage and Send an Email Notification if Usage Exceeds 80%

Outcome

A Bash script named monitor_memory.bash is created to monitor memory utilization on an Amazon Linux 2023 EC2 instance. If memory usage exceeds 80%, the script attempts to send an email notification to a configured recipient.

Memory utilization is monitored using a Bash script.

If memory usage exceeds 80%, an email notification is attempted.

Note: Email delivery requires a working mail-transfer service or SMTP relay. In the previous task, Postfix could not deliver directly to Gmail because the connection to port 25 timed out. Unless an SMTP relay is configured, the script may detect high memory usage without successfully delivering the email.

Objective

To automate memory monitoring and email alerting using Bash scripting. High memory utilization can cause applications to slow down, processes to be terminated by the Linux out-of-memory (OOM) mechanism, and services to become unstable.

This task demonstrates how to collect memory statistics, calculate memory utilization, compare it against a defined threshold, and trigger an email notification when the threshold is exceeded.

Prerequisites

- An AWS account with a running EC2 instance.
- Amazon Linux 2023 operating system.
- SSH access to the EC2 instance.
- Root privileges or appropriate permissions.
- Basic knowledge of Bash scripting and Linux commands.

- The free utility, usually provided by procps-ng.
- The mail command installed.
- A configured SMTP relay or working email delivery service.
- A valid recipient email address.

Step-by-Step Implementation

Step 1: Connect to the EC2 Instance

Connect to the Amazon Linux 2023 EC2 instance using SSH.

Verify the operating system:

cat /etc/os-release

Check the system memory:

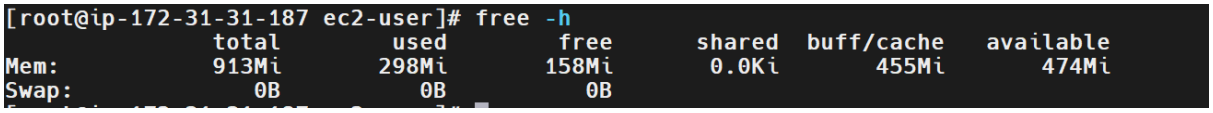
free -h

Example output:

	total	used	free	shared	buff/cache	available
Mem:	900Mi	250Mi	120Mi	10Mi	530Mi	500Mi
Swap:	0B	0B	0B			

The values above are illustrative. Your EC2 instance may show different values.

Screenshot Evidence



Step 2: Verify the Required Commands

Check whether the free command is available:

command -v free

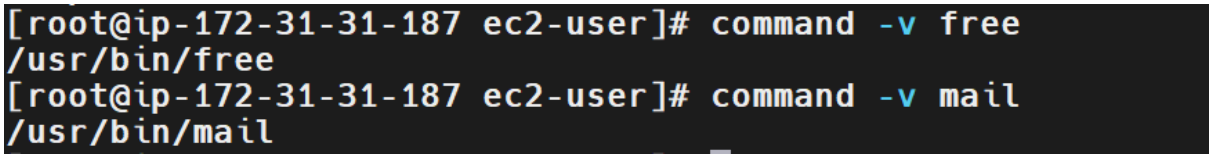
Check whether the mail command is available:

command -v mail

The free command is used to retrieve memory statistics. The mail command is used to submit an email notification.

Important: A path returned by command -v mail confirms only that the command is installed. It does not confirm that external email delivery is configured.

Screenshot Evidence



Step 3: Create the Bash Script

Create a file named monitor_memory.bash:

vim monitor_memory.bash

Press i and enter the following script:

```
#!/bin/bash

THRESHOLD=80
EMAIL="admin@example.com"

echo "Checking memory utilization..."

# Calculate memory usage percentage
MEMORY_USAGE=$(free | awk '/^Mem:/ {
   if ($2 > 0) {
      printf "%.2f", (($2 - $7) / $2) * 100
   }
}')

# Validate the calculated value
if ! [[ "$MEMORY_USAGE" =~ ^[0-9]+([.][0-9]+)?$ ]]; then
   echo "Error: Unable to determine memory usage."
   exit 1
fi

echo "Current Memory Usage: ${MEMORY_USAGE}%"
echo "Memory Threshold: ${THRESHOLD}%"

# Compare memory usage with the threshold
if awk -v usage="$MEMORY_USAGE" -v threshold="$THRESHOLD" \
   'BEGIN {exit !(usage > threshold)}'; then
```

```
    SUBJECT="High Memory Usage Alert"

    BODY="Warning: Memory usage on $(hostname) is ${MEMORY_USAGE}%, exceeding the threshold of ${THRESHOLD}%."

    if echo "$BODY" | mail -s "$SUBJECT" "$EMAIL"; then
        echo "High memory usage detected. Email command completed."
    else
        echo "Error: Email notification command failed."
        exit 1
    fi
else
    echo "Memory usage is within the configured threshold."
fi
```

Replace admin@example.com with your actual recipient email address.

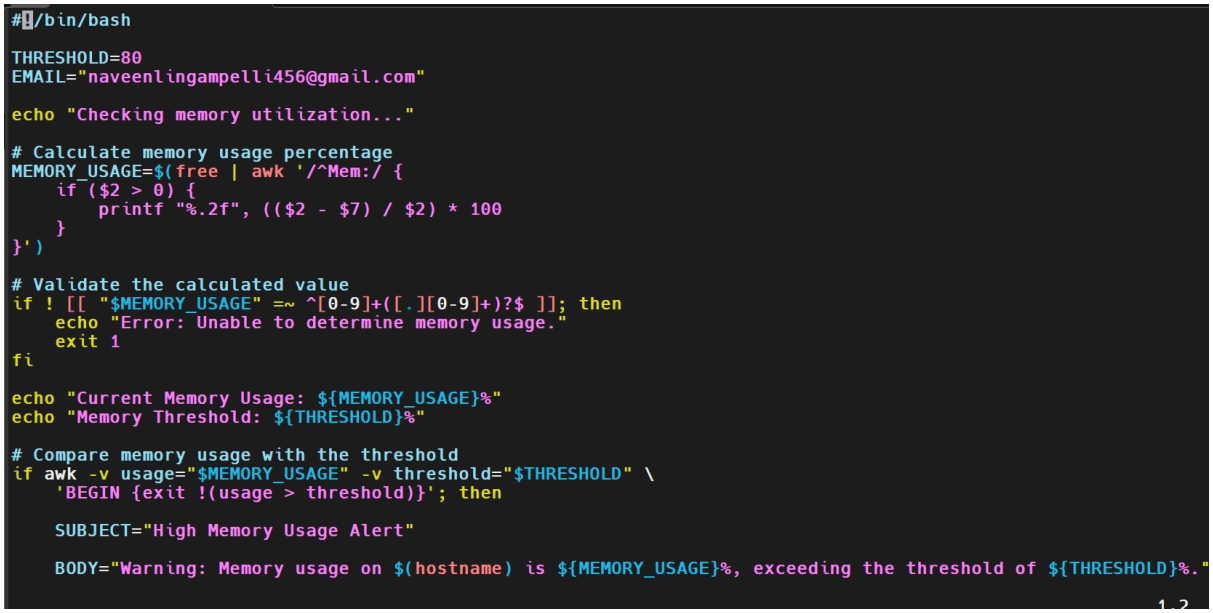
Save and exit Vim by pressing Esc, typing :wq, and pressing Enter.

Calculation used:

Memory Usage (%) = (Total Memory - Available Memory) / Total Memory × 100

This script uses the available-memory field reported by free to estimate memory utilization. This is generally more useful than simply subtracting the completely free memory from total memory because Linux also uses memory for buffers and cache.

Screenshot Evidence



```
#!/bin/bash

THRESHOLD=80
EMAIL="naveenlingampelli456@gmail.com"

echo "Checking memory utilization..."

# Calculate memory usage percentage
MEMORY_USAGE=$(free | awk '/^Mem:/ {
    if ($2 > 0) {
        printf "%.2f", (($2 - $7) / $2) * 100
    }
}')

# Validate the calculated value
if ! [[ "$MEMORY_USAGE" =~ ^[0-9]+([.][0-9]+)?$ ]]; then
    echo "Error: Unable to determine memory usage."
    exit 1
fi

echo "Current Memory Usage: ${MEMORY_USAGE}%"
echo "Memory Threshold: ${THRESHOLD}%"

# Compare memory usage with the threshold
if awk -v usage="$MEMORY_USAGE" -v threshold="$THRESHOLD" \
    'BEGIN {exit !(usage > threshold)}'; then

    SUBJECT="High Memory Usage Alert"

    BODY="Warning: Memory usage on $(hostname) is ${MEMORY_USAGE}%, exceeding the threshold of ${THRESHOLD}%."
```

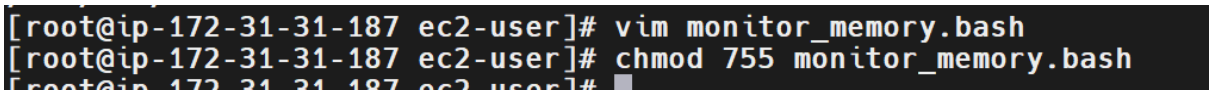
Step 4: Grant Execution Permission

Execute:

chmod 755 monitor_memory.bash

This grants the file owner read, write, and execute permissions, and other users read and execute permissions.

Screenshot Evidence



Step 5: Execute the Script

Run:

./monitor_memory.bash

If memory utilization is below or equal to 80%, the output should resemble:

Checking memory utilization...

Current Memory Usage: 45.00%

Memory Threshold: 80%

Memory usage is within the configured threshold.

If memory utilization exceeds 80%, the script attempts to submit an email notification.

Example output:

Checking memory utilization...

Current Memory Usage: 85.00%

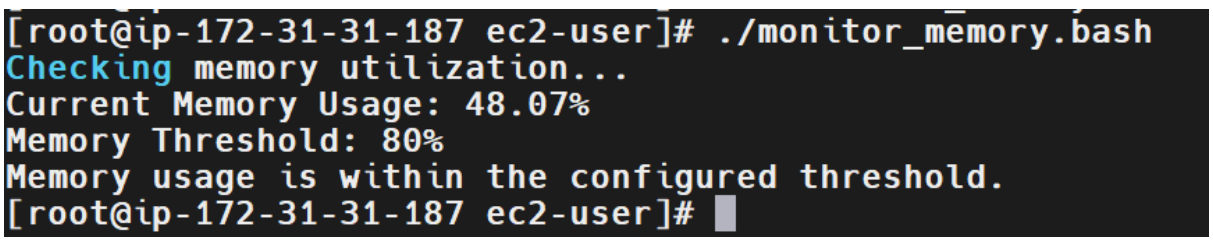
Memory Threshold: 80%

High memory usage detected. Email command completed.

The percentages are illustrative. Actual utilization depends on the current workload.

Note: The message Email command completed indicates that the local mail command returned success; it does not guarantee delivery to the recipient.

Screenshot Evidence



Step 6: Test the Alert Condition

On a lightly loaded EC2 instance, memory usage may never exceed 80%. You can test the alert logic by temporarily changing the threshold in the script to a low value, such as:

THRESHOLD=1

Run the script:

./monitor_memory.bash

If actual memory utilization exceeds 1%, the script attempts to send an email.

After testing, restore the intended threshold:

THRESHOLD=80

Important: Changing the threshold tests the conditional logic; it does not prove that the actual 80% alert has been triggered. To test high memory utilization, use a controlled workload in a practice environment and monitor the system carefully. Avoid exhausting memory on a production server.

Screenshot Evidence

Insert a screenshot showing the temporary test threshold and resulting output.

Step 7: Verify Email Delivery

If email delivery is configured, send a test message:

echo "Memory monitoring test email" | mail -s "Memory Alert Test" YOUR_EMAIL_ADDRESS

Replace YOUR_EMAIL_ADDRESS with the actual recipient address.

Check the inbox and the configured mail service's delivery logs.

If using Postfix, inspect recent logs:

journalctl -u postfix -n 50 --no-pager

If direct delivery to an external provider fails, configure an authenticated SMTP relay, such as Amazon SES. Installing Postfix and the mail command alone does not guarantee external email delivery.

Screenshot Evidence

```
[root@ip-172-31-31-187 ec2-user]# vim monitor_memory.bash
[root@ip-172-31-31-187 ec2-user]# ./monitor_memory.bash
Checking memory utilization...
Current Memory Usage: 48.45%
Memory Threshold: 1%
High memory usage detected. Email command completed.
[root@ip-172-31-31-187 ec2-user]#
```

Validation Steps

Step 1: Verify Memory Statistics

Execute:

free -h

Confirm that the system reports total, used, free, and available memory.

Step 2: Validate the Script Syntax

Run:

bash -n monitor_memory.bash

If no output appears, the script has passed Bash syntax checking.

Step 3: Verify the Threshold Logic

Confirm that the script attempts to send an email only when memory utilization is strictly greater than 80%.

Memory utilization	Expected behavior
40%	No email alert
80%	No email alert
85%	Email notification attempted

Step 4: Verify Email Delivery

Check the recipient's inbox or the configured email service's delivery logs. A successful local command alone is not sufficient proof of delivery.

Validation Screenshot

Insert screenshots showing the memory statistics, script execution, and email delivery evidence if available.

Explanation

The script uses the free command to retrieve memory statistics and awk to calculate memory utilization. It estimates used memory as total memory minus available memory, divides this value by total memory, and converts the result into a percentage.

The script compares the calculated percentage with the configured threshold of 80%. If memory utilization is greater than the threshold, it constructs an alert containing the hostname and current memory usage and submits it using the mail command. Otherwise, it reports that memory usage is within the configured limit.

Real-Time Use Case

DevOps engineers monitor memory utilization on application servers, database servers, and cloud instances to detect resource pressure. High memory usage may result from memory leaks, large workloads, excessive processes, or insufficient instance capacity.

Automated memory alerts help teams investigate performance problems before they lead to application failures. In production, this monitoring can be implemented through Amazon CloudWatch, Prometheus, or another observability platform with reliable alert delivery and dashboards.

Common Errors and Troubleshooting

Error	Cause	Resolution
free: command not found	The required utility is unavailable.	Verify the installation of procps-ng and the command path.
Memory usage cannot be calculated	Unexpected command output or invalid memory statistics.	Run free manually and inspect its output.
mail: command not found	The mail client is not installed.	Install an appropriate mail client and configure delivery.
/usr/sbin/sendmail: No such file or directory	A compatible mail-transfer agent is missing.	Install and configure an MTA or SMTP client.
Email is not received	SMTP relay, sender verification, or network connectivity is incorrect.	Check the mail service configuration and delivery logs.
Memory usage never exceeds 80%	The server is lightly loaded.	Use a controlled test threshold or a limited test workload.
Script permission denied	Execution permission is missing.	Run chmod 755 monitor_memory.bash.
Memory usage appears higher than expected	Linux memory accounting includes caches and buffers.	Use the available-memory-based calculation and inspect free -h.

Conclusion

The monitor_memory.bash script demonstrates how to collect memory statistics, calculate utilization, compare it against an 80% threshold, and attempt an email notification when the threshold is exceeded.

This task provides practical experience with Bash scripting, system monitoring, arithmetic calculations, conditional logic, and email alert integration. Reliable production monitoring requires a properly configured email delivery service and should ideally be combined with a dedicated monitoring platform.

Task 13: Configure Crontab Entries in Linux

Outcome

Successfully configure and verify seven different cron scheduling expressions on an Amazon Linux EC2 instance to automate tasks at specific dates, times, weekdays, and system reboot events.

Objective

To understand and implement cron scheduling in Linux using crontab. This exercise covers scheduling jobs for specific dates, particular weekdays, multiple times per day, daily execution, weekends, and system reboots.

Prerequisites

- An Amazon Linux EC2 instance.
- SSH access to the EC2 instance.
- A Linux user with permission to configure cron jobs.
- The cronie package installed.
- The crond service enabled and running.

Step-by-Step Implementation

Step 1: Verify Cron Installation and Service Status

Check whether the cron package is installed.

rpm -q cronie

If it is not installed, run:

sudo dnf install cronie -y

Start the cron service and enable it at boot:

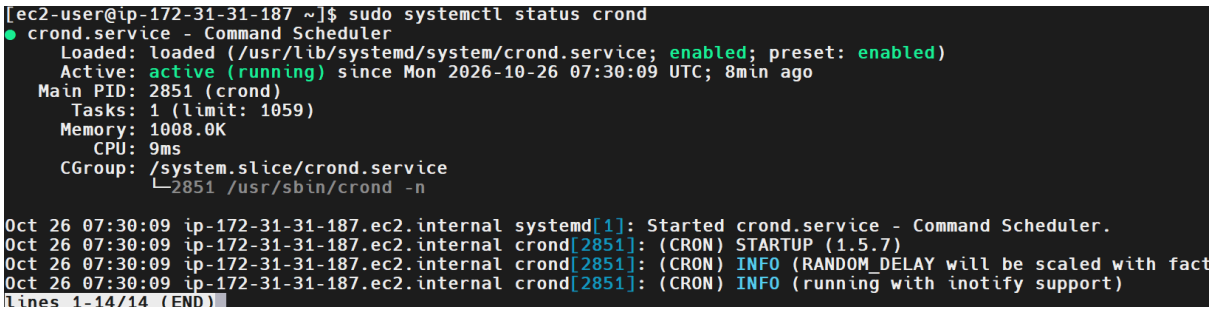
sudo systemctl enable --now crond

Verify the service status:

sudo systemctl status crond

Expected Result: The crond service should show active (running).

Screenshot Evidence:



Step 2: Open the Crontab Editor

Open the current user's crontab:

crontab -e

If prompted, select your preferred text editor. Add the required cron entries in the following steps.

Note: Cron jobs execute with the permissions and environment of the user whose crontab contains the entry. Use sudo crontab -e only when the jobs must run as root.

Step 3: Schedule a Job for April 5th at Midnight

Requirement: Run a job at 12:00 AM on April 5th every year.

Cron expression:

0 0 5 4 *

Example entry:

0 0 5 4 * echo "April 5 midnight job executed" >> /tmp/cron_task13.log

Explanation:

- 0 — minute 0.
- 0 — hour 0, or midnight.
- 5 — fifth day of the month.
- 4 — April.
- * — every day of the week.

Screenshot Evidence: Capture the crontab entry and its output when displayed with crontab -l.

Step 4: Schedule a Job on the 5th of November, January, and June Only When It Is Thursday

Requirement: Run a job on the 5th day of November, January, and June when the date falls on Thursday.

Cron expression:

0 0 5 11,1,6 4

Example entry:

0 0 5 11,1,6 4 echo "Thursday date-specific job executed" >> /tmp/cron_task13.log

Important: Standard cron implementations generally treat the day-of-month and day-of-week fields as an OR when both are restricted. Therefore, the expression above does **not** reliably mean “the 5th of these months only if it is Thursday”; it can run on every Thursday in the listed months as well as on the 5th of those months.

To enforce the exact requirement, use a daily check and test both the date and weekday:

0 0 * 11,1,6 * ["$(date +\%d)" = "05"] && ["$(date +\%u)" = "4"] && echo "Thursday date-specific job executed" >> /tmp/cron_task13.log

Here, date +%u returns 4 for Thursday. The backslashes before % are required in crontab commands.

Screenshot Evidence: Capture the cron entry and a terminal showing the date/weekday condition used.

Step 5: Schedule a Job at the 5th and 27th Minutes of Hours 9, 10, and 11 Every Day

Requirement: Run the job at 9:05 AM, 9:27 AM, 10:05 AM, 10:27 AM, 11:05 AM, and 11:27 AM every day.

Cron expression:

5,27 9,10,11 * * *

Example entry:

5,27 9,10,11 * * * echo "Scheduled time job executed" >> /tmp/cron_task13.log

Explanation:

- 5,27 — at minutes 5 and 27.
- 9,10,11 — during the 9th, 10th, and 11th hours.
- The remaining * fields mean every day, every month, and every weekday.

Screenshot Evidence: Capture the expression and verify it appears in crontab -l.

Step 6: Schedule a Job at 9:34 AM on August 15th

Requirement: Run the job at 9:34 AM on August 15th every year.

Cron expression:

34 9 15 8 *

Example entry:

34 9 15 8 * echo "August 15 job executed" >> /tmp/cron_task13.log

Explanation:

- 34 — minute 34.
- 9 — 9 AM.
- 15 — the 15th day of the month.
- 8 — August.
- * — any day of the week.

Screenshot Evidence: Capture the cron expression in the crontab.

Step 7: Schedule a Job Every Midnight

Requirement: Run a job every day at 12:00 AM.

Cron expression:

0 0 * * *

Example entry:

0 0 * * * echo "Daily midnight job executed" >> /tmp/cron_task13.log

Explanation: The job runs at minute 0 of hour 0 every day.

Screenshot Evidence: Capture the cron entry and its presence in the crontab listing.

Step 8: Schedule a Job Every Saturday at 11:59 PM

Requirement: Run the job every Saturday night at 11:59 PM.

Cron expression:

59 23 * * 6

Example entry:

59 23 * * 6 echo "Saturday night job executed" >> /tmp/cron_task13.log

Explanation:

- 59 — minute 59.
- 23 — hour 23, or 11 PM.
- * * — every day of the month and every month.
- 6 — Saturday on standard Linux cron implementations.

Screenshot Evidence: Capture the Saturday scheduling expression.

Step 9: Schedule a Job After Every Reboot

Requirement: Execute a command whenever the system reboots.

Cron expression:

@reboot

Example entry:

@reboot echo "System reboot job executed" >> /tmp/cron_task13.log

Explanation: The @reboot special string tells cron to run the command when the cron service starts during system startup. It does not run at a recurring clock time.

Screenshot Evidence: Capture the @reboot entry. If you test it, rebooting the EC2 instance can interrupt your SSH session; reconnect afterward and inspect the log.

Step 10: Save and Verify All Entries

After adding the entries, save and exit the editor. List the configured jobs:

crontab -l

For easier verification, the seven example entries can be placed in the same user's crontab. Each entry appends to /tmp/cron_task13.log.

Check the log file if it exists:

cat /tmp/cron_task13.log

Note: The log file is created only after an entry executes or you manually create it. Listing a cron entry does not prove that its scheduled time has occurred.

Screenshot Evidence: Capture the complete output of crontab -l.

Validation Steps

1. Confirm that the cron package is installed:

rpm -q cronie

2. Confirm that the cron service is running:

systemctl is-active crond

Expected output:

active

3. Verify the current user's configured cron jobs:

crontab -l

4. Inspect the log file after jobs have executed:

cat /tmp/cron_task13.log

5. Check cron service logs for execution or scheduling issues:

sudo journalctl -u crond --since today --no-pager

Validation Screenshot:

```
[ec2-user@ip-172-31-31-187 ~]$ sudo systemctl is-active crond
active
[ec2-user@ip-172-31-31-187 ~]$ sudo crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-31-187 ~]$ sudo crontab -l
0 0 * * * echo "Midnight job executed" >> /tmp/cron_task13.log
0 0 5 11,1,6 4 echo "Thursday date-specific job executed" >> /tmp/cron_task13.log
5,27 9,10,11 * * * echo "Scheduled time job executed" >> /tmp/cron_task13.log
34 9 15 8 * echo "August 15 job executed" >> /tmp/cron_task13.log
0 0 * * * echo "Daily midnight job executed" >> /tmp/cron_task13.log
59 23 * * 6 echo "Saturday night job executed" >> /tmp/cron_task13.log
@reboot echo "System reboot job executed" >> /tmp/cron_task13.log
```

Explanation

Cron is a time-based job scheduler available on Linux systems. A cron expression consists of five fields: minute, hour, day of the month, month, and day of the week. These fields allow administrators to automate recurring maintenance activities, backups, monitoring scripts, log cleanup, and other routine operations without manually running commands.

The @reboot directive is a special cron scheduling string that runs a command when the cron service starts during system startup. Cron jobs should be tested carefully because they run with the configured user's permissions and may have a more limited environment than an interactive shell. Absolute paths are recommended for commands and scripts. In addition, cron uses the server's configured timezone, so verify the timezone when the schedule depends on a specific local time.

Real-Time Use Case

In a production environment, cron can automate daily database backups, periodic log rotation, cleanup of temporary files, scheduled health checks, and recurring monitoring scripts. For example, a disk-monitoring script can be scheduled to run every day and write its findings to a log or send an alert when a threshold is exceeded. An @reboot entry can start a lightweight initialization script after system startup.

Common Errors and Troubleshooting

Issue	Possible Cause	Solution
Cron jobs do not execute	crond is stopped	Run sudo systemctl enable --now crond.
No crontab exists	No jobs have been saved for the current user	Run crontab -e and add the required entries.
Job runs under the wrong permissions	Entry is configured for a different user	Check whether the entry belongs in crontab -e or sudo crontab -e.
Job executes at an unexpected time	Server timezone differs from the expected timezone	Check with timedatectl and adjust the timezone or schedule.
A conditional date/weekday job runs unexpectedly	Both day-of-month and day-of-week fields are restricted	Use a command-level date/weekday check as shown in Step 4.
Log file is missing	The scheduled time has not occurred, or the command failed	Check the cron service logs and verify the command manually.
Command works manually but not in cron	Cron has a limited environment or cannot locate a command	Use absolute command paths and redirect errors to a log file.
@reboot command does not appear to run	Cron service startup or command timing issue	Inspect journalctl -u crond and use an absolute path for the command.

Conclusion

In this exercise, seven cron scheduling scenarios were covered: a specific date and time, a date restricted to a weekday, multiple minute/hour combinations, a yearly date-specific job, daily midnight execution, a Saturday-night schedule, and execution after reboot. These examples demonstrate how Linux cron can automate recurring administrative tasks and system operations.

Final Verification: Ensure all entries are saved, confirm crond is active, and retain screenshots of the configured crontab and validation commands. Remember that scheduled execution should be verified from actual logs when the relevant time occurs.